

Trajectory Soup: Pushing the Compute-Scaling Frontier of LLM Mid-training via Diverse Trajectories

Zhehao Huang^{1,2*†}, Changxin Tian^{1*}, Qingyuan Yang¹, Kunlong Chen¹, Ziqi Liu¹, Zhiqiang Zhang^{1†}, Xiaolin Huang^{2†}, Jun Zhou¹

¹Ling Team, Ant Group, ²Shanghai Jiao Tong University

*Equal contribution, [†]Contribution during internship at Ant Group, [‡]Corresponding author

Mid-training equips pretrained large language models with specialized and reasoning capabilities, but the returns of this stage are bounded since additional serial compute yields little further downstream improvement and can even degrade some capabilities, which places a practical ceiling on how much compute mid-training absorbs. We revisit how this compute should be allocated to a single run or multiple similar optimizations. We find that branches forked from a shared checkpoint under various controlled recipe reaches measurably different regions of parameter space, and establish a form of compatible diversity that extending one run cannot supply. Therefore, we introduce **Trajectory Soup**, which distributes a mid-training budget over several independent branches, and consolidates strongest checkpoints selected on validation through intra- and inter-trajectory averaging into a single model. A local bias and variance analysis separates the two averaging levels, showing that inter-trajectory averaging removes residual error beyond the reach of averaging within a trajectory, while checkpoint selection carries a bias that bounds how many checkpoints are worth merging. Across model scales, learning-rate schedules, token budgets, and trajectory counts, Trajectory Soup improves aggregate downstream performance over the strongest single-trajectory average under matched budgets and keeps improving as budgets expand, with the advantage preserved after an identical post-training pipeline. These results position trajectory allocation and merging as a practical way to extend the compute-scaling frontier of mid-training beyond serial saturation.

Date: September 30, 2026

Correspondence: {tianchangxin.tcx, lingyao.zzq}@antgroup.com
{kinght_h, xiaolinhuang}@sjtu.edu.cn

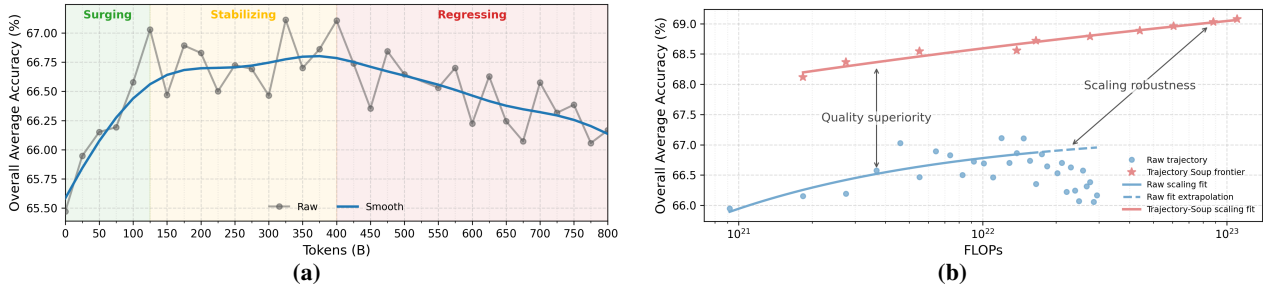


Figure 1 Motivation and compute-allocation overview of Trajectory Soup. (a) Performance saturation along serial mid-training trajectories. (b) Compute scaling of raw training and Trajectory Soup. Overall Average Accuracy is plotted against cumulative training FLOPs on a logarithmic scale.

1 Introduction

Mid-training plays an important role in developing specialized and reasoning capabilities in pre-trained large language models (LLMs) (Liu et al., 2026; Shao et al., 2024). However, unlike in pretraining, increasing mid-training compute does not always lead to better downstream performance. As illustrated in Fig. 1a, downstream performance improves rapidly early in training and then saturates, further training can even degrade capabilities. These observations expose a practical *scaling wall* in mid-training: beyond a certain point, extending the single training run brings little downstream improvement. This motivates us to explore alternative ways of allocating mid-training compute to sustain downstream gains.

To this end, we consider distributing compute across multiple shorter trajectories. Starting from a common pretrained checkpoint, independent branches can follow different optimization paths through controlled variation of the training recipe (Frankle et al., 2020), potentially producing complementary updates even on the same data distribution. These potentially complementary updates can then be consolidated into a single model through model merging. However, existing methods typically focus on either intra-trajectory merging, which averages checkpoints within a single run (Izmailov et al., 2018), or inter-trajectory merging, which combines runs through endpoint averaging (Wortsman et al., 2022) or periodic aggregation (Douillard et al., 2023). Whether combining these two forms of merging can make compute allocation across trajectories more effective remains underexplored. Inspired by the compute-allocation perspective of scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022), we study how to divide a mid-training budget between trajectory count and trajectory length. This leads us to ask: *whether exploiting both at once turns a fixed compute budget spread over several trajectories into a single model that outperforms a longer serial run?*

Our empirical observations and theoretical analysis support this possibility. First, after within-trajectory averaging, the examined branches retain distinct dominant directions. Interpolating between their averaged anchors further reduces validation loss, indicating that useful differences remain after temporal averaging. Second, a local bias–variance analysis separates within-trajectory fluctuations from persistent branch variation and characterizes how checkpoint selection trades variance reduction against mean displacement. Building on these findings, we propose **Trajectory Soup**, which combines both stages. It forks several independent branches from a shared pretrained checkpoint and trains each for the same number of tokens. Within each branch, it ranks checkpoints by validation set and averages a fixed number of the lowest-loss checkpoints into a trajectory anchor. It then averages these anchors into a single model, retaining the original architecture, tokenizer, and inference procedure. We evaluate two budget settings: budget reallocation, where branches share a fixed total token budget, and budget expansion, where we add branches while keeping the per-branch token budget fixed.

Trajectory Soup turns compute into downstream gains where serial training no longer does. Under a matched token budget, Trajectory Soup surpasses the strongest intra-trajectory average as well as inter-trajectory baselines that keep only trajectory endpoints or that average the entire pooled set of checkpoints. Expanding the budget with additional trajectories widens the margin instead of exhausting it. Fig. 1b shows that Trajectory Soup frontier lies above the raw scaling curve at every matched compute level, and it keeps rising at a steady rate while the raw fit bends over and its checkpoints scatter downward as the budget grows. The frontier also improves as trajectories are added, and each trajectory needs to contribute only its strongest few checkpoints, so broad coverage of the loss basin rather than deep sampling within it carries the gain. The advantage persists after supervised fine-tuning, so scaling trajectories pays off end to end rather than only at

the mid-training checkpoint. In summary, our contributions are threefold.

- We recast compute allocation for LLM mid-training as a question of how many optimization trajectories to run rather than how long to run a single one, and identify trajectory count as an allocation axis that remains productive after the length of a single trajectory has saturated. Trajectories separated only by controlled variation of the recipe stay geometrically distinct, and interpolating between them improves on both endpoints.
- We introduce Trajectory Soup, which selects the strongest checkpoints inside each trajectory and averages the resulting anchors across trajectories, yielding a single model with unchanged inference cost. A local bias and variance analysis grounds this design by separating what the two averaging levels contribute, quantifying the selection-dependent bias, showing that uniform weights minimize the variance term, and predicting a finite preferred checkpoint count that decreases as trajectories are added.
- We validate the method across model scales, learning-rate schedules, training budgets, and trajectory counts, show that the advantage is retained after a shared post-training pipeline, and isolate trajectory diversity from checkpoint sampling density as the source of the gains.

2 Empirical Motivation: Saturation and Complementary Trajectories

Serial mid-training exhibits diminishing downstream returns. Our scaling baseline fixes the model architecture and a high-quality mid-training corpus and adopts the best configuration found in hyperparameter search. As [Fig. 1a](#) shows, downstream performance first improves with the token budget, then plateaus, and eventually declines. The serial-training recipe therefore reaches a practical saturation regime beyond which further tokens stop paying off. Two mechanisms plausibly contribute to this decline. With model capacity fixed, prolonged exposure to a limited corpus offers diminishing returns, consistent with findings on the decreasing value of repeated training tokens ([Muennighoff et al., 2023](#)). Separately, stochastic fluctuations along the optimization path can leave a checkpoint displaced from nearby parameter regions of lower validation loss ([Mandt et al., 2017](#)). The second mechanism can be mitigated by weight averaging ([Zhou et al., 2026](#)), as the raw and averaged checkpoints of [Fig. 3](#) indicate. However, once local fluctuations are largely attenuated, additional checkpoints contribute less noise reduction. Averaging within a single trajectory therefore runs into diminishing returns.

The limitation of extending the same run motivates us to allocate additional compute to several branches instead. The remainder of this section tests two prerequisites for that allocation: (1) whether controlled recipe perturbations produce distinct optimization directions, and (2) whether combining the resulting branch anchors reaches lower validation loss.

Intra-trajectory averaging reveals partially overlapping branch directions. We first examine the directional diversity across branches that are comparably optimized. We parallelly run five trajectories under various recipes (denoted as EXP1 $\sim$ EXP5). Each branch forks from the baseline run and perturbed by a controlled recipe change, such as changing the data order, learning rate, batch size, learning-rate schedule, or optimizer. All branches satisfy the training-loss screening criterion in [Sec. 3.1](#), controlling differences in measured optimization quality. Now we try to discover whether these branches explore distinct optimization directions. [Zhou et al. \(2026\)](#) report that averaging can reveal an approximately rank-one structure within a late-stage training trajectory. Following this observation, we smooth each branch through intra-trajectory averaging, extract its leading principal component, and compute pairwise cosine similarities with other trajectories.



Figure 2 Compatible branch directions overlap. Pairwise cosine similarity between leading directions for five compatible branches. All pairs only have weak overlap and partial optimization similarity.

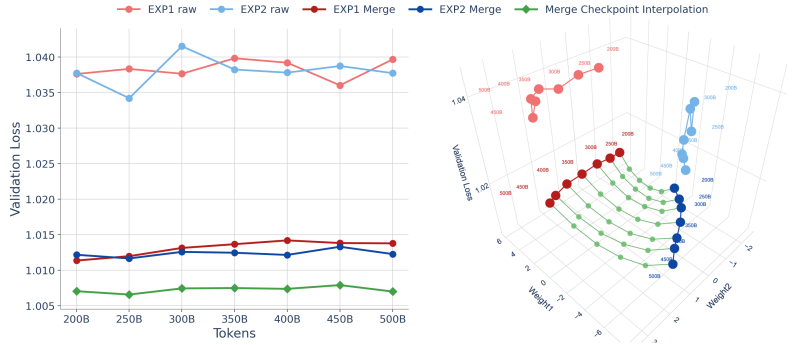


Figure 3 Validation loss within and across trajectories. Left: validation loss against consumed tokens for the raw checkpoints of EXP1 and EXP2, for their within-trajectory merges, and for the best interpolation between the two merged anchors. Right: the same models in a two-dimensional projection of the weight space, with one interpolation path per token budget.

Fig. 2 shows positive but limited alignment among these branches, indicating that controlled recipe perturbations produce distinct dominant directions with partial overlap. Directional diversity alone, however, does not establish a merging benefit, which depends on the validation-loss geometry between the resulting anchors rather than on the angle between them. Detailed protocol of trajectory geometry construction is provided in Appendix C.

Inter-trajectory averaging reaches lower validation loss. We assess the benefit of combining intra-trajectory averaged anchors through linear interpolation (Frankle et al., 2020). At each matched token horizon, we take the averaged checkpoints of two screened branches and sweep the interpolation coefficient between them, following the protocol in Appendix C. Fig. 3 shows inter-trajectory interpolation achieves lower validation loss than intra-trajectory averaging alone. The right panel illustrates the corresponding geometry: branches departing from a shared pretrained checkpoint follow distinct optimization directions, while interpolation between their averaged anchors reaches a lower-loss region. This coexistence of directional diversity and compatibility under parameter averaging is consistent with prior findings on linear mode connectivity and model soups (Frankle et al., 2020; Wortsman et al., 2022). Together, the direction and interpolation analyses identify distinct optimization paths whose progress can be combined beneficially. These findings motivate allocating additional compute to multiple compatible branches as serial training approaches saturation. Sec. 3 formalizes this construction as Trajectory Soup.

3 Trajectory Soup

3.1 Problem Formulation and Budget Accounting

Let θ_0 be a shared pre-trained checkpoint and let $\mathcal{P}$ be the target training distribution, which we keep common to all branches. Branch $n \in \{1, \dots, N\}$ follows recipe ψ_n and produces parameters $\theta_n(i)$ after consuming i mid-training tokens. All branches retain the same architecture, so their parameters can be combined coordinate by coordinate. For a common per-branch budget t , the total token budget is $T = Nt$. Each recipe ψ_n is obtained by modifying the best configuration found in the baseline hyperparameter search along one or more of its data order, learning rate, batch size, learning-rate schedule, or optimizer. Because such perturbations can also destabilize a run, we admit a branch only when a compatibility observable $\mathcal{J}$ stays within a tolerance ε , that is, when $\mathcal{J}(\theta_n(t)) \leq \varepsilon$. In our implementation $\mathcal{J}$ is the relative training-loss gap to the baseline at the same token horizon and $\varepsilon = 0.01$. This screening keeps the branches comparable in optimization quality,

Algorithm 1 Trajectory Soup

Require: Shared checkpoint θ_0
Require: Training distribution $\mathcal{P}$
Require: Screened recipes $\{\psi_n\}_{n=1}^N$
Require: Branch horizon t
Require: Checkpoint count K

- 1: **for** $n = 1, \dots, N$ **do**
- 2: Train branch n from θ_0 on $\mathcal{P}$ under ψ_n for t tokens, saving checkpoints at $\mathcal{C}_n(t)$
- 3: Rank $\mathcal{C}_n(t)$ by ascending validation loss
- 4: $\mathcal{I}_n(t, K) \leftarrow$ the first K positions
- 5: $\bar{\theta}_n(t, K) \leftarrow K^{-1} \sum_{i \in \mathcal{I}_n(t, K)} \theta_n(i)$
- 6: **end for**
- 7: $\theta_{\text{Traj-Soup}}(N, t, K) \leftarrow N^{-1} \sum_{n=1}^N \bar{\theta}_n(t, K)$
- 8: **return** $\theta_{\text{Traj-Soup}}(N, t, K)$

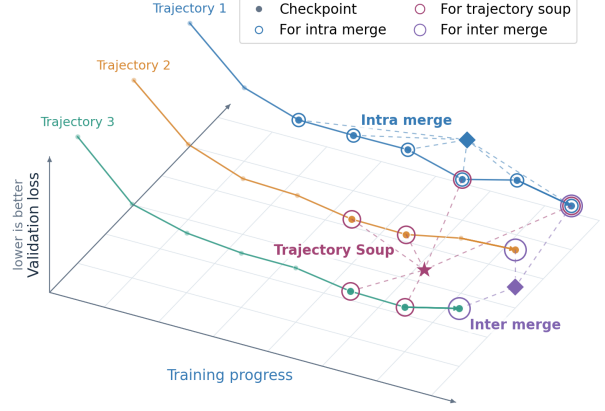


Figure 4 Trajectory Soup diagram. Three branches are shown across training progress, trajectory index, and validation loss.

while their parameter-space directions remain diverse. Changes to the data mixture are a possible extension, and we leave their effects to future work.

3.2 Trajectory Soup

The observations in **Sec. 2** suggest merging at two levels: within a branch where averaging attenuates local fluctuations along one path, and across branches where averaging consolidates directions that individual paths do not share. Trajectory Soup performs these two stages in order.

Step 1: Intra-Trajectory Merging: selecting and averaging checkpoints within each branch.

Let $\mathcal{C}_n(t)$ be the candidate token positions of checkpoints saved from branch n by t . Checkpoints beyond this horizon are excluded. We rank the candidates of each branch by validation loss $\hat{\mathcal{L}}_{\text{val}}(\theta)$, keep the best K positions, and average them into a branch anchor, so that within-trajectory variation is attenuated before any branch is combined with another,

$$\mathcal{I}_n(t, K) \in \arg \min_{\mathcal{I} \subseteq \mathcal{C}_n(t), |\mathcal{I}|=K} \sum_{i \in \mathcal{I}} \hat{\mathcal{L}}_{\text{val}}(\theta_n(i)), \quad \bar{\theta}_n(t, K) = \frac{1}{K} \sum_{i \in \mathcal{I}_n(t, K)} \theta_n(i). \quad (1)$$

Step 2: Inter-Trajectory Merging: combining branch anchors. Trajectory Soup averages the selected branch anchors,

$$\theta_{\text{Traj-Soup}}(N, t, K) = \frac{1}{N} \sum_{n=1}^N \bar{\theta}_n(t, K) = \frac{1}{NK} \sum_{n=1}^N \sum_{i \in \mathcal{I}_n(t, K)} \theta_n(i). \quad (2)$$

Fig. 4 illustrates the procedure and places it beside the two single-stage merges it generalizes. Algorithm 1 states the two averaging stages for the admitted branches. The two-stage construction is algebraically equivalent to uniformly averaging the selected $M = NK$ checkpoints and the step order can be reversed. The special case $N = 1$ recovers selected intra-trajectory averaging, while $K = 1$ averages the best checkpoint from each branch. Trajectory Soup introduces branch count as an additional axis of compute allocation. As serial training approaches saturation, we seek to distribute the budget across shorter compatible trajectories, whose complementary progress is consolidated into a single model. **Fig. 1b** further shows that allocating additional compute to complementary branches sustains downstream gains beyond the observed serial-saturation regime.

4 Theoretical Insights

We theoretically analyze how checkpoint selection and the two averaging stages in Trajectory Soup affect validation loss. We use a local quadratic model (Mandt et al., 2017) and standard bias–variance identities (Bishop, 2006) to separate selection-dependent bias, persistent branch variation, and within-trajectory fluctuations.

4.1 A Local Validation-Loss Model

Let $\mathcal{L}_Q(\theta) = \mathbb{E}_{z \sim Q}[\ell(\theta, z)]$ denote population validation loss, estimated by $\hat{\mathcal{L}}_{\text{val}}$ for checkpoint ranking in Eq. (1). We assume that all analyzed checkpoints and averages lie in a region $\mathcal{B}$ around a stationary local minimizer θ^* , where $\mathcal{L}_Q(\theta) = \mathcal{L}^* + \frac{1}{2}\|\theta - \theta^*\|_H^2$ with $\mathcal{L}^* = \mathcal{L}_Q(\theta^*)$, $H = \nabla^2 \mathcal{L}_Q(\theta^*) \succeq 0$, and $\|x\|_H^2 = x^\top H x$. The compatibility screen of Sec. 3.1 and the interpolation profiles of Sec. 2 motivate this local model, which underpins the stochastic analysis of constant-step SGD (Mandt et al., 2017) and the flatness interpretation of weight averaging (Wortsman et al., 2022).

Theorem 4.1 (Local bias and variance decomposition). *For any random model ϑ supported in $\mathcal{B}$ with finite second moments,*

$$\mathbb{E}[\mathcal{L}_Q(\vartheta)] - \mathcal{L}^* = \frac{1}{2}\|\mathbb{E}[\vartheta] - \theta^*\|_H^2 + \frac{1}{2}\text{tr}(H \text{Cov}(\vartheta)). \quad (3)$$

The two terms measure displacement of the mean model and variation around it, respectively. Checkpoint selection can change both. The identity requires only finite moments and local compatibility, and makes no Gaussian or optimizer-specific assumption. The proof is given in Appendix B.1.

4.2 Branchwise Decomposition of Validation Loss

We first identify the part of a branch’s loss that intra-trajectory merging can reach. Let $\theta_{n,(j)}$ be the checkpoint of validation rank j on branch n , and let $\bar{\theta}_n$ represent the checkpoint that averages its top- K selection in Eq. (1). Let $\mathcal{F}_n$ represent the persistent component of branch randomness, the part that a single run fixes and then carries along its entire path, so that conditioning on $\mathcal{F}_n$ holds that component fixed while retaining the variation among selected checkpoints. Theorem 4.2 splits the loss of the anchor into the displacement of its mean, the within-trajectory variation that survives conditioning, and the persistent variation it inherits from $\mathcal{F}_n$.

Theorem 4.2 (Branchwise decomposition of validation loss). *Under the conditions of Theorem 4.1,*

$$\mathbb{E}[\mathcal{L}_Q(\bar{\theta}_n)] - \mathcal{L}^* = \underbrace{\frac{1}{2}\|\mathbb{E}[\bar{\theta}_n] - \theta^*\|_H^2}_{B_n(K)} + \underbrace{\frac{1}{2}\mathbb{E}[\text{tr}(H \text{Cov}(\bar{\theta}_n \mid \mathcal{F}_n))]}_{V_{\text{intra},n}(K)} + \underbrace{\frac{1}{2}\text{tr}(H \text{Cov}(\mathbb{E}[\bar{\theta}_n \mid \mathcal{F}_n]))}_{V_{\text{inter},n}(K)}. \quad (4)$$

All three contributions, $B_n(K)$, $V_{\text{intra},n}(K)$, and $V_{\text{inter},n}(K)$ are nonnegative. The bias $B_n(K)$ collects an offset shared by every branch, which neither averaging stage removes, together with offsets specific to the recipe ψ_n and to the selection itself. Appendix A.1 records that decomposition and the random-effects construction behind it. The two variance terms differ in what can reach them: $V_{\text{intra},n}(K)$ is precisely what merging within a branch acts on, whereas $V_{\text{inter},n}(K)$ is invisible to any operation confined to a single trajectory. Sampling the path more densely within one trajectory may reveal more about that single branch, but it gives no access to the cross-branch variation. This tradeoff explains how the intra-trajectory gains of Sec. 2 can diminish, and how fast that term falls is what Sec. 4.3 quantifies. Appendix B.2 proves the theorem.

Method	Intra	Inter	Expected excess loss in the local quadratic model
Rank-1 checkpoint	No	No	$B_n(1) + V_{\text{intra}} + V_{\text{inter}}$
Intra-trajectory merging	Yes	No	$B_n(K) + V_{\text{intra}}/K_{\text{eff}}(K) + V_{\text{inter}}$
Inter-trajectory merging	No	Yes	$B_{\text{soup}}(1) + V_{\text{intra}}/N + V_{\text{inter}}/N$
Trajectory Soup	Yes	Yes	$B_{\text{soup}}(K) + V_{\text{intra}}/(NK_{\text{eff}}(K)) + V_{\text{inter}}/N$

Table 1 Comparative analysis of averaging operations under the local quadratic model and the homogeneous independent-anchor assumptions of Theorem 4.3.

4.3 Hierarchical Variance Reduction

To quantify the residual reduction, note that the term $V_{\text{intra},n}(K)$ of Eq. (4) is the curvature-weighted covariance of the average, over the K selected ranks, of the fluctuations of the selected checkpoints around their conditional means given $\mathcal{F}_n$. Temporal correlation and ranking-induced dependence therefore both limit how fast it decays. For branches with a common rank-1 residual contribution $V_{\text{intra}} = V_{\text{intra},n}(1) > 0$, we write $V_{\text{intra},n}(K) = V_{\text{intra}}/K_{\text{eff}}(K)$, whose effective count is a variance ratio: it reaches K only when those fluctuations are uncorrelated with equal marginal contributions, and stays below K whenever their pairwise contributions are nonnegative. Appendix A.1 gives the residual covariance behind this ratio together with its bounds and limiting cases. Thus $K_{\text{eff}}(K)$ settles the first averaging level, which operates inside a single branch. The second level averages the N branch anchors themselves. The following theorem therefore introduces the corresponding bias term and collects both levels in one expression.

Theorem 4.3 (Hierarchical variance reduction). *Suppose the shared-shift condition holds and the selected anchors are independent under the fixed experimental setup. Define $B_{\text{soup}}(K) = \frac{1}{2} \left\| \frac{1}{N} \sum_{n=1}^N \mathbb{E}[\bar{\theta}_n] - \theta^* \right\|_H^2$ with common contributions $V_{\text{intra},n}(K) = V_{\text{intra}}/K_{\text{eff}}(K)$ and $V_{\text{inter},n}(K) = V_{\text{inter}}$. The equally averaged model $\theta_{\text{Traj-Soup}} = N^{-1} \sum_{n=1}^N \bar{\theta}_n$ of Eq. (2) satisfies*

$$\mathbb{E}[\mathcal{L}_Q(\theta_{\text{Traj-Soup}})] - \mathcal{L}^* = B_{\text{soup}}(K) + \frac{V_{\text{intra}}}{NK_{\text{eff}}(K)} + \frac{V_{\text{inter}}}{N} \quad (5)$$

The bias $B_{\text{soup}}(K)$ depends on the chosen branch collection. Temporal averaging reduces the intra-trajectory contribution through $K_{\text{eff}}(K)$, while branch averaging reduces both stochastic contributions through N . Tab. 1 compares these effects. Appendix B.3 gives the proof.

Variance optimality of uniform averaging. Finally, the same decomposition reveals that the two *uniform* averages of Trajectory Soup are variance-optimal rather than merely convenient. Consider replacing them with deterministic nonnegative weights over ranks and branches, each normalized to sum to one. Under the condition that every selected checkpoint of a branch shares the same aggregate curvature-weighted covariance with its selected set, and that the branch-level collections are independent, no such reweighting can attain a smaller stochastic contribution than $V_{\text{intra}}/(NK_{\text{eff}}(K)) + V_{\text{inter}}/N$ in Eq. (5), and the uniform temporal and branch weights of Eq. (2) attain this bound. This condition permits correlated residuals and is therefore weaker than independence, but it is substantive, since equal marginal variances and positive correlations alone do not imply equal covariance sums.

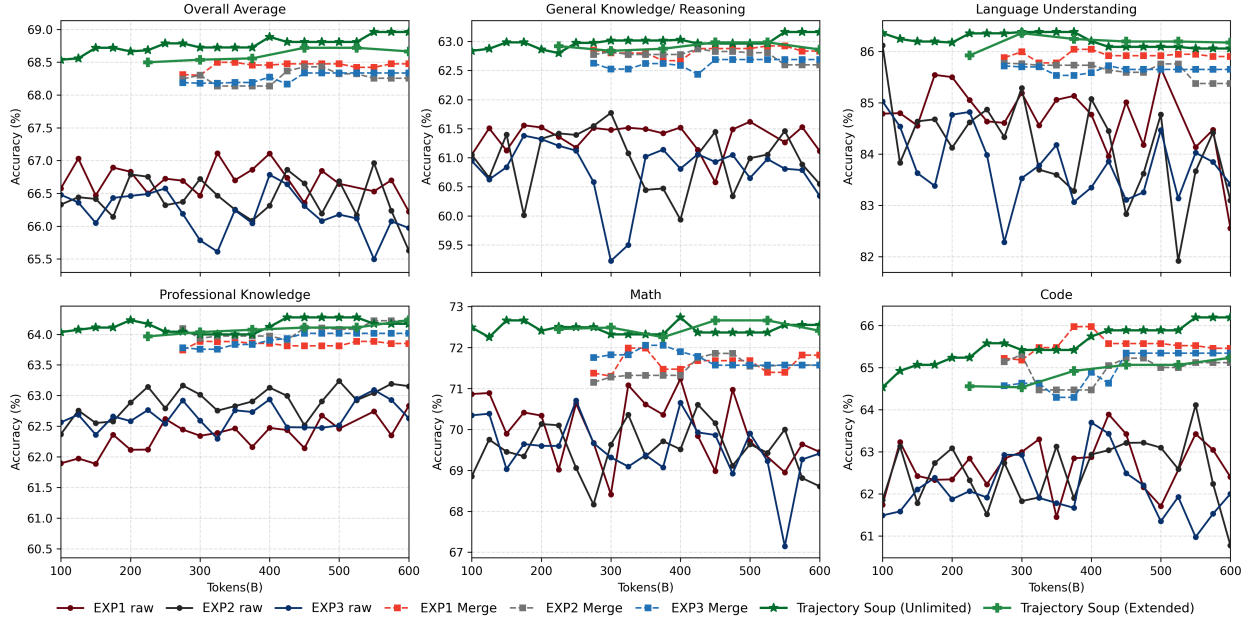


Figure 5 Performance of different mid-training trajectories and merging strategies as the training-token budget increases. Panels report the overall average accuracy and the five capability categories. Branch identities, merge sizes, and the Limited and Extended budget conventions follow Sec. 5.1.

5 Experiments

5.1 Experimental Setup

Model and mid-training trajectory settings. We conduct our default experiments with Ling-3.0-Tiny (inclusionAI, 2026), a sparse MoE language model with 7.9B total and 1.3B parameters activated per token. We fork multiple independent trajectories from the same checkpoint. To induce controlled trajectory diversity, each branch differs from the default configuration along one or more of the following dimensions: data-shuffling seed, peak learning rate, global batch size, learning-rate schedule, and optimizer. The default branch horizon is $t = 600$ B tokens, which is sufficiently long for mid-training performance to approach saturation. Every branch saves a checkpoint each 25B tokens. Appendices D record the architecture, training recipe, and evaluation protocol.

Merging baselines. Given a candidate pool, we compare four merging scopes: (i) **Single EXP Merge**, which combines checkpoints within a single experiment run (EXP); (ii) **Model Soup**, which combines the final checkpoint of each trajectory; (iii) **Full Soup**, which applies no selection and naively averages all checkpoints pooled from multiple trajectories, and which we therefore report as a selection ablation in Sec. 5.3.2; and (iv) our **Trajectory Soup**, which jointly selects and combines checkpoints both within and across trajectories.

Compute-budget comparisons. We distinguish compute-matched evaluation from compute scaling. In the **Limited** setting, N trajectories share a total budget of $T = Nt$ tokens (each covering T/N tokens) and are compared compute-matched against a single trajectory trained for the same T . Reported budgets denote aggregate consumption. In the **Extended** setting, we merge N fully trained trajectories and increase N . Reported budgets denote per-trajectory consumption, testing whether merging converts an expanded budget ($t \rightarrow Nt$ tokens) into improved quality.

Table 2 Performance comparison of different checkpoint-merging strategies on the mid-training evaluation suite. Each row reports the best evaluated configuration of its strategy; the corresponding checkpoint counts are recorded in Appendix D.5.

Base Model	General Knowledge & Reasoning	Language Modeling	Professional Knowledge	Math	Code	Overall Average
Single-Trajectory Merge	62.80	85.86	63.70	72.33	65.36	68.55
Model Soup (Limited)	62.74	85.80	63.59	72.53	64.74	68.43
Model Soup (Extended)	62.78	85.95	63.68	72.49	65.78	68.67
Trajectory Soup (Limited)	62.99	86.19	64.11	72.66	65.07	68.72
Trajectory Soup (Extended)	63.16	86.06	64.17	72.55	66.19	68.96

Table 3 Post-training performance after applying the same SFT procedure to mid-training checkpoints produced by different merging strategies. The merging protocols are those of Tab. 2.

Instruct Model	Math	Code	Knowledge	Reasoning	Instruction Following	Function Call	Overall Average
Single-Trajectory Merge	68.30	46.58	70.51	62.96	59.46	50.67	61.11
Model Soup (Limited)	68.50	47.59	69.36	63.56	58.79	51.52	61.05
Model Soup (Extended)	68.63	46.76	69.65	64.41	59.29	51.63	61.23
Trajectory Soup (Limited)	69.44	48.33	69.81	63.60	58.88	51.49	61.39
Trajectory Soup (Extended)	68.53	47.50	69.26	64.48	60.38	52.85	61.52

5.2 Main Results of Trajectory Soup

Intra- and Inter-trajectory Merging Effects during Mid-training. We systematically compare checkpoint-merging strategies during mid-training, focusing on how intra- and inter-trajectory effects interact across three independently trained trajectories: EXP1 (Baseline), EXP2 (Data Shuffle), and EXP3 (Hyperparameter Change). Tab. 2 summarizes the best candidate produced by each strategy. Inter-trajectory diversity yields further gains over intra-trajectory merging. In the compute-matched setting, Trajectory Soup (Limited) outperforms the strongest single-trajectory merge and Model Soup (Limited). In the full-trajectory setting, Trajectory Soup (Extended) achieves the best overall average of 68.96, exceeding the strongest single-trajectory merge and Model Soup (Extended). These results suggest that the gain cannot be attributed solely to either trajectory diversity or the number of merged checkpoints. Instead, effective scaling requires jointly exploiting inter-trajectory complementarity and selectively filtering checkpoints within each trajectory. The complete performance curves over increasing training-token budgets are deferred to Fig. 5, which shows that the ordering below holds throughout mid-training.

Long-term Implications for Post-training. The preceding experiments establish that Trajectory Soup achieves the strongest aggregate performance at the mid-training stage, which naturally raises the question of whether this advantage persists after post-training. To investigate this, we use the mid-training checkpoints produced by each merging strategy as base models, apply an identical SFT recipe, and evaluate the post-training capabilities, with results reported in Tab. 3. The advantage of Trajectory Soup is largely preserved after SFT, indicating that our method provides a stronger initialization whose benefits transfer to subsequent post-training.

Table 4 Comparison of single-trajectory merging and cross-trajectory Trajectory Soup during mid-training of the smaller 2B-parameter MoE model. The Limited and Extended budget conventions follow Sec. 5.1.

Model	General Knowledge & Reasoning	Language Modeling	Professional Knowledge	Math	Code	Overall Average
Single-Trajectory Merge	47.71	72.85	43.41	54.21	35.96	49.55
Trajectory Soup (Limited)	47.88	73.32	43.70	54.13	35.77	49.64
Trajectory Soup (Extended)	48.35	73.98	44.37	54.87	35.51	50.07

Table 5 Ablation of merging coefficients and checkpoint-selection strategies. Every row follows the Trajectory Soup (Extended) setting of Tab. 2, with Baseline the default configuration of Sec. 5.1.

Ablation	Coefficient	Selection	General Knowledge & Reasoning	Language Modeling	Professional Knowledge	Math	Code	Overall Average
Baseline	EQUAL	Top-K each	63.16	86.06	64.17	72.55	66.19	68.96
Coefficient	ISQRT	Top-K each	63.22	85.87	64.21	72.26	66.00	68.85
	RANK	Top-K each	63.12	85.91	64.16	72.24	65.94	68.80
	RSQRT	Top-K each	63.04	86.03	64.26	72.59	65.97	68.89
Selection	EQUAL	Global top-NK	62.99	86.24	64.18	72.46	65.43	68.76
	EQUAL	Tail-K per branch	62.78	84.97	64.29	71.77	65.25	68.35
	EQUAL	All checkpoints	62.94	85.88	64.26	71.79	65.70	68.60

5.3 Empirical Analysis of Trajectory Soup

5.3.1 Robustness across models and learning-rate schedules.

To examine whether the benefits of Trajectory Soup depend on a particular base model or learning-rate schedule, we repeat our experiments on a smaller 2B-parameter MoE model with 32 experts, activating 8 experts per token, using a warmup-stable-decay (WSD) learning-rate schedule (Hu et al., 2024). We compare intra-trajectory averaging within each trajectory against our Trajectory Soup. For each strategy, Tab. 4 reports the merging results. These results reproduce the aggregate trend observed under our default configuration. Under the compute-matched Limited setting, Trajectory Soup achieves an overall average of 49.64, slightly outperforming the strongest intra-trajectory baseline, despite each constituent trajectory receiving only half of the total token budget. This suggests that the complementarity introduced by trajectory diversity can compensate for reduced training-token coverage along each individual trajectory. Under the Extended setting, Trajectory Soup further improves the overall average to 50.07, exceeding the best intra-trajectory result. Together with the results obtained under the default setup, these findings indicate that the gains from Trajectory Soup are not confined to a single model scale or scheduling choice, but remain reproducible on a smaller MoE model trained with WSD.

5.3.2 Ablation Studies on the Trajectory Soup Configuration

We ablate three key design choices in Trajectory Soup: (1) how merge coefficients are assigned, (2) how checkpoints are selected, and (3) how the checkpoint budget is allocated across trajectories.

Effect of merge coefficients. We compare uniform averaging against three rank-based weighting schemes, detailed in Appendix D.6. As shown in Tab. 5, none of them improves upon the equal-weight baseline. Once checkpoint quality is controlled through selection, more elaborate coefficient design provides little additional benefit, so uniform averaging is the strongest evaluated scheme and avoids introducing additional hyperparameters.

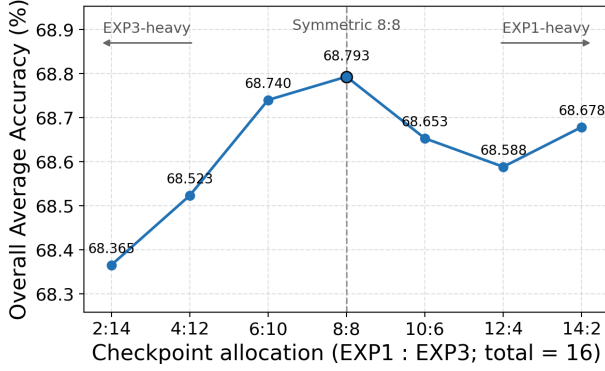


Figure 6 Effect of asymmetric merging allocation on Trajectory Soup performance.

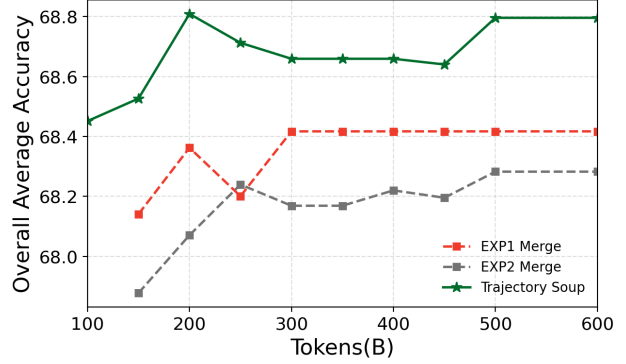


Figure 7 Performance of densely sampled single run merging and equally sized sparse Trajectory Soup.

Effect of checkpoint selection. We next fix merge coefficients to be uniform and consider three alternative selection strategies, also defined in Appendix D.6. All three underperform the default *Top-K each* strategy. These results identify both checkpoint quality and balanced trajectory representation as important selection criteria.

Importance of Balanced Allocation across Trajectories. Fig. 6 tests balanced representation directly: we fix the total number of merged checkpoints to 16 and vary their allocation between EXP1 and EXP3 from 2:14 to 14:2. Performance follows an approximately inverted-U-shaped profile centered on the balanced allocation, with the symmetric 8:8 configuration attaining the highest overall average. Sufficiently balanced participation from both trajectories is therefore necessary to capture their complementary information, supporting symmetric allocation as a robust default.

Overall, the three ablations consistently indicate that the effectiveness of Trajectory Soup is driven by the combination of trajectory diversity, checkpoint quality, and balanced representation, rather than by sophisticated coefficient design. Among the configurations evaluated, the simple combination of uniform averaging, top-K selection within each trajectory, and symmetric cross-trajectory allocation provides the strongest aggregate performance while requiring minimal additional tuning.

5.3.3 Trajectory Diversity over Sampling Density

The preceding experiments show that Trajectory Soup consistently outperforms intra-trajectory merging throughout mid-training. However, this advantage could conceivably arise from a trivial confound: *Trajectory Soup may simply draw from a larger checkpoint pool, thereby offering more candidates or a higher effective sampling density.* To isolate the contribution of trajectory diversity, we conduct a controlled comparison in which both the candidate-pool size and the number of merged checkpoints are strictly matched. Specifically, we fix the merge size to $M = 12$ and compare densely sampled intra-trajectory merging with sparsely sampled cross-trajectory merging. In our experiment, EXP1 Merge and EXP2 Merge sample checkpoints every 12.5B tokens along their respective trajectories and uniformly average the top 12 checkpoints. In contrast, Trajectory Soup halves the per-trajectory sampling density by sampling checkpoints every 25B tokens from each trajectory and uniformly averages the top six checkpoints from each, again totaling 12. Thus, at a per-trajectory token coverage of t , both settings draw from exactly $t/12.5\text{B}$ candidate checkpoints and merge the same number of checkpoints, and the only systematic difference is whether these candidates originate from a single trajectory or from two independently optimized ones. As shown in Fig. 7, Trajectory Soup outperforms both intra-trajectory baselines at every comparable token budget, demonstrating

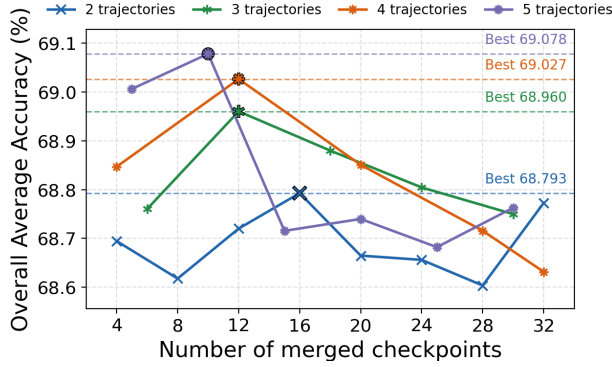


Figure 8 Scaling of multi-trajectory Top-K Trajectory-Soup with the number of merged checkpoints.

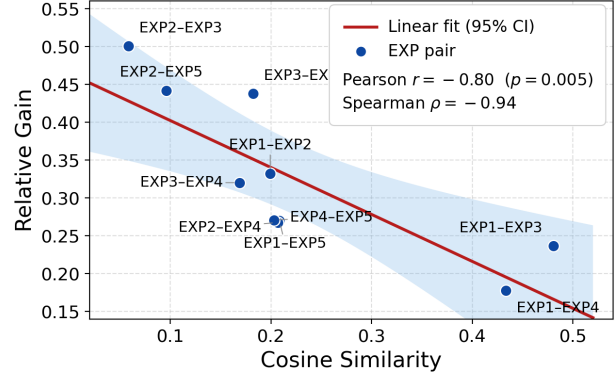


Figure 9 Weaker directional alignment predicts larger interpolation gains.

that its gains cannot be attributed to a larger candidate pool or denser checkpoint sampling. This controlled comparison therefore provides direct evidence that the gains of Trajectory Soup stem from complementary information accumulated along independently evolved trajectories, rather than from increased checkpoint availability.

5.3.4 Scaling the Number of Trajectories and Merged Checkpoints

We further investigate how Trajectory Soup scales along two axes: the number of participating trajectories N and the number of top-ranked checkpoints K contributed by each. For every (N, K) configuration we uniformly average the top K checkpoints of each trajectory, and **Fig. 8** reports the Overall Average against the total number of merged checkpoints, revealing two patterns. First, *trajectory diversity, rather than checkpoint depth, drives performance*: the best attainable score rises monotonically from 68.79 with two trajectories to 69.08 with five, while the increments steadily shrink. This is consistent with the standard interpretation of weight averaging, where uniform averaging cancels trajectory-specific errors only when the merged solutions reside in a shared low-loss basin while making quasi-independent mistakes, so a new trajectory helps most when it expands coverage of the basin, and its marginal value fades as the covered region saturates. Second, *the soup benefits most from being selective*: all groups peak within a narrow window of roughly ten to sixteen merged checkpoints, beyond which enlarging the pool brings oscillation or steady decline, with the four-trajectory soup eventually falling below the two-trajectory optimum. This makes Trajectory Soup practical to scale, adding trajectories reliably raises the performance ceiling, and retaining only a handful of top checkpoints per trajectory is sufficient to nearly reach it.

5.3.5 Directional Diversity Predicts Interpolation Gain

The scaling results show that adding trajectories improves the merged model, but they do not identify *which* branches contribute the improvement or why. We therefore test whether pairwise branch geometry predicts the benefit of merging. **Fig. 9** plots the cosine similarity of the two leading directions against the relative gain of interpolating the pair’s endpoints on the overall evaluation score. Both quantities are defined in Appendix C. The association is strongly negative and nearly monotone, with Pearson $r = -0.80$ ($p = 0.005$) and Spearman $\rho = -0.94$: the near-orthogonal pairs gain the most and the most aligned pairs the least. Directional diversity therefore explains the scaling trend above and supplies a cheaply computable criterion, available before any merging is performed, for selecting which branches to merge.

6 Related Work

6.1 Mid-Training and Horizon-Extensible Scaling

Classical language-model scaling laws characterize the dependence of performance on model size, training data, and compute. Compute-optimal training rules balance parameter count and token consumption (Hoffmann et al., 2022). In data-constrained regimes, the effective value of repeated tokens decreases as repetition grows (Muennighoff et al., 2023). These results motivate distinguishing additional token consumption from the useful progress obtained from it. Training schedules provide flexibility in extending a run. MiniCPM introduces the Warmup–Stable–Decay schedule, whose stable phase supports continued training followed by a cooldown (Hu et al., 2024). The river-valley analysis of WSD explains how a large learning rate can support progress along a slowly varying direction while maintaining fluctuations in sharper directions (Wen et al., 2024). Complementing this optimization perspective, mid-training studies examine how mixtures of general and specialized data improve the starting point for post-training (Liu et al., 2026). Our study adds trajectory count to the allocation problem and examines selective weight-space aggregation under both matched and expanded training budgets.

6.2 Checkpoint Averaging Along a Single Trajectory

Stochastic Weight Averaging combines iterates from constant or cyclical learning-rate training and often reaches flatter solutions with improved generalization (Izmailov et al., 2018). Trainable Weight Averaging instead learns the averaging coefficients by optimizing in the low-dimensional subspace spanned by the candidate checkpoints rather than fixing them in advance (Li et al., 2023b). In language-model pre-training, Sanyal et al. (2023) show that averaging can improve convergence and downstream performance, with larger gains under high learning rates and sufficiently spaced checkpoints. Pre-trained Model Averaging studies checkpoint merging across dense and MoE language models and relates stable-phase averages to annealing behavior (Li et al., 2025). WSM develops the connection through the effective coefficients assigned to historical updates (Tian et al., 2026). Extra-Merge identifies approximately one-dimensional structure in late-stage merged trajectories and uses this structure for extrapolation (Zhou et al., 2026). These approaches establish the value of temporal aggregation. We build on this foundation by combining selected temporal averages across independently evolved branches and studying how this changes the allocation of training compute.

6.3 Combining Multiple Training Trajectories

Prediction ensembles retain multiple models at inference time. Weight-space combinations can consolidate compatible models into one parameter set. Model Soups demonstrates effective averaging of models fine-tuned from a common initialization with different hyperparameters (Wortsman et al., 2022). In language modeling, Branch-Train-Merge trains domain-specialized expert language models that can be ensembled or averaged, and studies their performance under controlled training cost (Li et al., 2022). Branch-Train-MiX incorporates independently trained feedforward components into MoE layers, followed by training that learns routing (Sukhbaatar et al., 2024). The geometry underlying such combinations is related to mode connectivity. Garipov et al. (2018) construct low-loss curves between trained solutions, while Frankle et al. (2020) study linear connectivity under different realizations of SGD noise after a shared training history. Our interpolation analysis examines the straight segments between mid-training anchors, which are directly relevant to arithmetic parameter averaging.

7 Discussion and Limitations

Scope of the compute accounting. Our allocation comparison holds processed training tokens fixed at a common architecture and sequence length, which isolates the allocation question but charges nothing for the machinery around it. Checkpoint storage, the validation passes used for ranking, and the search over how many trajectories and checkpoints to combine all consume resources that a complete development-cost account would attribute to each strategy, and accelerator utilization differs between one long run and several concurrent shorter ones. Making these costs explicit is the natural next step, because a cost model that prices a trajectory against a token would let the allocation be chosen by optimization rather than by sweeping candidates.

What the analysis assumes. The theory is exact under a local quadratic model of the validation loss and inherits whatever error that model carries over the region spanned by the selected checkpoints and their averages. The geometric diagnostics are complementary but weaker. Direction angles characterize differences in a chosen parameter-space representation, and interpolation profiles probe particular averages, so neither establishes that the region containing all branches is jointly well behaved. Both are also measured only after the branches exist. A more useful account would predict which recipe perturbations generate error components that averaging can remove, turning diversity from a property screened after training into one designed before it.

Where averaging stops paying. Averaging cannot remove error that the branches share, so strongly correlated trajectories leave a floor that no amount of merging crosses, and perturbations chosen too aggressively push branches into regions where the merged model is worse than its members. The same tension governs the budget, since at a fixed total more branches means shorter runs, and admitting more checkpoints per branch eventually admits weak ones. Trajectory Soup therefore depends on selecting compatible and sufficiently trained branches together with a suitable per-branch checkpoint count, which we currently fix by validation search. Learning this allocation during training, or adapting it as branches diverge, would remove the remaining hand-tuning and is where we expect the largest gains to come from.

8 Conclusion

Mid-training has so far been scaled by making a single optimization trajectory longer, and that axis eventually stops paying. This paper treats the number of trajectories as an allocation variable of the same kind as parameters and tokens, and works out what it takes to spend a budget that way. Trajectory Soup forks independent branches from a shared checkpoint, selects the strongest checkpoints inside each branch, and consolidates the resulting anchors into a single model whose architecture, tokenizer, and inference cost are unchanged. A local bias and variance analysis accounts for the design, showing why the two averaging levels address different parts of the error, why uniform weights are the natural default at both levels, and why checkpoint selection carries a bias that bounds how many checkpoints are worth merging. Weight-space consolidation is a general way to convert parallel exploration into one deliverable model, rather than a technique specific to mid-training. The same structure arises wherever a budget can be split across runs that share an initialization, including continued pretraining, domain adaptation, and the increasingly long post-training pipelines that follow. Realizing that generality calls for understanding which perturbations create removable error, how to price a trajectory against a token, and how far compatibility survives objectives that carry models far from their shared origin. Treating trajectory count as a standard dimension of the compute-scaling question, rather than a special case of model merging, is in our view the more productive framing for the work that follows.

References

- Aider. Aider code editing benchmarks, 2026.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Jun Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *ArXiv*, abs/2108.07732, 2021.
- Lucas Bandarkar, Davis Liang, Benjamin Muller, Mikel Artetxe, Satya Narayan Shukla, Don Husa, Naman Goyal, Abhinandan Krishnan, Luke Zettlemoyer, and Madian Khabsa. The belebele benchmark: a parallel reading comprehension dataset in 122 language variants. In *Annual Meeting of the Association for Computational Linguistics*, 2024.
- Youssef Bencheikroun, Megi Dervishi, Mark Ibrahim, Jean-Baptiste Gaya, Xavier Martinet, Grégoire Mialon, Thomas Scialom, Emmanuel Dupoux, Dieuwke Hupkes, and Pascal Vincent. Worldsense: A synthetic benchmark for grounded reasoning in large language models. *ArXiv*, abs/2311.15930, 2023.
- Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *AAAI Conference on Artificial Intelligence*, 2020.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mo Bavarian, Clemens S. Winter, Phil Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgens Guss, Alex Nichol, Igor Babuschkin, Suchir Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457, 2018.
- Karl Cobbe, Vineet Kosaraju, Mo Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *ArXiv*, abs/2110.14168, 2021.
- Arthur Douillard, Qixiang Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc’Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. Diloco: Distributed low-communication training of language models. *ArXiv*, abs/2311.08105, 2023.
- Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *International Conference on Machine Learning*, 2020.
- Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry Vetrov, and Andrew Gordon Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns. In *Neural Information Processing Systems*, 2018.
- Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida Wang. Cruxeval: A benchmark for code reasoning, understanding and execution. In *International Conference on Machine Learning*, 2024.
- Yancheng He, Shi-Long Li, Jiaheng Liu, Yingshui Tan, Wei-Xun Wang, Hui Huang, Xing-Yuan Bu, Hangyu Guo, Chengwei Hu, Bo Zheng, Zhuoran Lin, Xuepeng Liu, Dekai Sun, Shirong Lin, Zhicheng Zheng, Xiaoyong Zhu, Wenbo Su, and Bo Zheng. Chinese simpleqa: A chinese factuality evaluation for large language models. *ArXiv*, abs/2411.07140, 2024.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *ArXiv*, abs/2009.03300, 2020.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *ArXiv*, abs/2103.03874, 2021.
- HMMT. Harvard-mit mathematics tournament, 2026.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and L. Sifre. Training compute-optimal large language models. *ArXiv*, abs/2203.15556, 2022.

- Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zhen Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chaochao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. Minicpm: Unveiling the potential of small language models with scalable training strategies. *ArXiv*, abs/2404.06395, 2024.
- Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Fanchao Qi, Yao Fu, Maosong Sun, and Junxian He. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. *ArXiv*, abs/2305.08322, 2023.
- inclusionAI. Ling-3.0-tiny. <https://huggingface.co/inclusionAI/Ling-3.0-tiny>, 2026.
- Pavel Izmailov, Dmitrii Podoprikin, T. Garipov, Dmitry P. Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In *Conference on Uncertainty in Artificial Intelligence*, 2018.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, 2025.
- Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *ArXiv*, abs/1705.03551, 2017.
- Jared Kaplan, Sam McCandlish, Thomas Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeff Wu, and Dario Amodei. Scaling laws for neural language models. *ArXiv*, abs/2001.08361, 2020.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur P. Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc V. Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 2019.
- Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard H. Hovy. Race: Large-scale reading comprehension dataset from examinations. *ArXiv*, abs/1704.04683, 2017.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Venkatesh Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. In *Neural Information Processing Systems*, 2022.
- Haonan Li, Yixuan Zhang, Fajri Koto, Yifei Yang, Hai Zhao, Yeyun Gong, Nan Duan, and Tim Baldwin. Cmmlu: Measuring massive multitask language understanding in chinese. In *Findings of the Association for Computational Linguistics*, 2024a.
- Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiayi Yang, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Chenhao Ma, Kevin C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In *Neural Information Processing Systems*, 2023a.
- Margaret Li, Suchin Gururangan, Tim Dettmers, Mike Lewis, Tim Althoff, Noah A. Smith, and Luke Zettlemoyer. Branch-train-merge: Embarrassingly parallel training of expert language models. *ArXiv*, abs/2208.03306, 2022.
- Qintong Li, Leyang Cui, Xueliang Zhao, Lingpeng Kong, and Wei Bi. Gsm-plus: A comprehensive benchmark for evaluating the robustness of llms as mathematical problem solvers. *ArXiv*, abs/2402.19255, 2024b.
- Tao Li, Zhehao Huang, Qinghua Tao, Yingwen Wu, and X. Huang. Trainable weight averaging: Efficient training by optimizing historical solutions. In *International Conference on Learning Representations*, 2023b.
- Wenhao Li, Fanchao Qi, Maosong Sun, Xiaoyuan Yi, and Jiarui Zhang. Ccpm: A chinese classical poetry matching dataset. *ArXiv*, abs/2106.01979, 2021.
- Yunshui Li, Yiyuan Ma, Shen Yan, Chaoyi Zhang, Jing Liu, Jianqiao Lu, Ziwen Xu, Mengzhao Chen, Minrui Wang, Shiyi Zhan, Jin Ma, Xunhao Lai, Deyi Liu, Yao Luo, Xingyan Bin, Hongbin Ren, Mingji Han, Wenhao Hao, Bairen Yi, Lingjun Liu, Bole Ma, Xiaoying Jia, Xun Zhou, Siyuan Qiao, Liang Xiang, and Yonghui Wu. Model merging in pre-training of large language models. *ArXiv*, abs/2505.12082, 2025.
- Emmy Liu, Graham Neubig, and Chenyan Xiong. Midtraining bridges pretraining and posttraining distributions. In *International Conference on Machine Learning*, 2026.

- Hongwei Liu, Zi-Long Zheng, Yu Qiao, Haodong Duan, Zhiwei Fei, Fengzhe Zhou, Wenwei Zhang, Songyang Zhang, Dahua Lin, and Kai Chen. Mathbench: Evaluating the theory and application proficiency of llms with a hierarchical mathematics benchmark. *ArXiv*, abs/2405.12209, 2024.
- Jiawei Liu, Chun Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In *Neural Information Processing Systems*, 2023.
- Kaijing Ma, Xinrun Du, Yun-Ran Wang, Haoran Zhang, Zhoufutu Wen, Xingwei Qu, Jian Yang, Jiaheng Liu, Ming-Hao Liu, Xiang Yue, Wenhao Huang, and Ge Zhang. Kor-bench: Benchmarking language models on knowledge-orthogonal reasoning tasks. *ArXiv*, abs/2410.06526, 2024.
- Stephan Mandt, Matthew D. Hoffman, and David M. Blei. Stochastic gradient descent as approximate bayesian inference. *J. Mach. Learn. Res.*, 2017.
- Mathematical Association of America. American invitational mathematics examination (AIME), 2026.
- Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. *ArXiv*, abs/2305.16264, 2023.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph Gonzalez. The berkeley function calling leaderboard (bfc1): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning*, 2025.
- Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hanna Hajishirzi. Generalizing verifiable instruction following. In *Neural Information Processing Systems*, 2025.
- Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for squad. *ArXiv*, abs/1806.03822, 2018.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *ArXiv*, abs/2311.12022, 2023.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. *ArXiv*, abs/1907.10641, 2019.
- Sunny Sanyal, Atula Neerkaje, Jean Kaddour, Abhishek Kumar, and Sujay Sanghavi. Early weight averaging meets high learning rates for llm pre-training. *ArXiv*, abs/2306.03241, 2023.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Jun-Mei Song, Mingchuan Zhang, Y. K. Li, Yu Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *ArXiv*, abs/2402.03300, 2024.
- Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners. *ArXiv*, abs/2210.03057, 2022.
- Sainbayar Sukhbaatar, Olga Golovneva, Vasu Sharma, Hu Xu, Xi Victoria Lin, Baptiste Rozière, Jacob Kahn, Shang-Wen Li, Wen tau Yih, Jason E. Weston, and Xian Li. Branch-train-mix: Mixing expert llms into a mixture-of-experts llm. *ArXiv*, abs/2403.07816, 2024.
- Mirac Suzgun, Nathan Scales, Nathanael Scharli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics*, 2023.
- Zhengyang Tang, Xingxing Zhang, Benyou Wang, and Furu Wei. Mathscale: Scaling instruction tuning for mathematical reasoning. In *International Conference on Machine Learning*, 2024.
- M-A-P Team, Xinrun Du, Yi-Fan Yao, Kaijing Ma, Bingli Wang, Tianyu Zheng, King Zhu, Minghao Liu, Yiming Liang, Xiaolong Jin, Zhen-Nan Wei, Chujie Zheng, Kaixin Deng, Shawn Gavin, Shian Jia, Sichao Jiang, Yiyan Liao, Rui Li, Qinrui Li, Sirun Li, Yizhi Li, Yunwen Li, David Ma, Yuansheng Ni, Haoran Que, Qiyao Wang, Zhoufutu Wen, Si-Yuan Wu, T. Russell Hsing, Ming Xu, Zhen-Zhu Yang, Ze Wang, Juntong Zhou, Yuelin Bai, Xing-Yuan Bu, Chenglin Cai, Liang Chen, Yifan Chen, Chengtuo Cheng, Tianhao Cheng, Keyi Ding, Si-Min Huang, Yun-Jing Huang, Yaoru Li, Yi-Zhe Li, Zhaoqun Li, Tianhao Liang, Chengdong Lin, Hongquan Lin, Yi Bo Ma, Tian-Tian Pang, Zhongyuan Peng, Zifan Peng, Qige Qi, Shi Qiu, Xingwei Qu, Shanghaoran Quan, Yizhou Tan, Zili Wang, Chenqing Wang, Hao Wang, Yiya Wang, Yubo Wang, Jiajun Xu, Kexin Yang, Ru-Qing Yuan, Yuanhao Yue, Tianyang Zhan, Chun Zhang, Jinyang Zhang, Xiyue Zhang, Xingjian Zhang, Yue Zhang, Yong-Chi Zhao, Xiangyu Zheng, Chenghua Zhong, Yang Gao, Zhoujun Li, Dayiheng Liu, Qian Liu, Tianyu Liu, Shiwen Ni, Junran Peng, Yujia Qin, Wenbo Su, Guoyin Wang, Shi Wang, Jian Yang, Min Yang, Mengxuan Cao, Xiang Yue, Zhaoxiang Zhang, Wangchunshu Zhou, Jiaheng Liu, Qunshu

- Lin, Wenhao Huang, and Ge Zhang. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. In *Neural Information Processing Systems*, 2025.
- Changxin Tian, Jiapeng Wang, Qian Zhao, Kunlong Chen, Jia Liu, Zi-Qi Liu, Jiaxin Mao, Wayne Xin Zhao, Zhi-Qiang Zhang, and Jun Zhou. Wsm: Decay-free learning rate schedule via checkpoint merging for llm pre-training. In *International Conference on Learning Representations*, 2026.
- Min-Yang Tian, Luyu Gao, Shizhuo Dylan Zhang, Xinan Chen, Cun-Wei Fan, Xuefei Guo, Roland Haas, Pan Ji, Kittithat Krongchon, Yao-Hui Li, Shengyan Liu, Di Luo, Yu-Tao Ma, Hao Tong, Kha Trinh, Chenyu Tian, Zihan Wang, Bohao Wu, Yanyu Xiong, Sheng Yin, Min Zhu, Kilian Adriano Lieret, Yanxin Lu, Genglin Liu, Yufeng Du, Tianhua Tao, Ofir Press, Jamie Callan, Eliu A. Huerta, and Hao Peng. Scicode: A research coding benchmark curated by scientists. In *Neural Information Processing Systems*, 2024.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max W.F. Ku, Kai Wang, Alex Zhuang, Rongqi "Richard" Fan, Xiang Yue, and Wenhua Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *ArXiv*, abs/2406.01574, 2024.
- Jason Wei, Karina Nguyen, Hyung Won Chung, Yunxin Joy Jiao, Spencer Papay, Amelia Glaese, John Schulman, and William Fedus. Measuring short-form factuality in large language models. *ArXiv*, abs/2411.04368, 2024.
- Tianwen Wei, Jian Luan, W. Liu, Shuang Dong, and Bin Wang. Cmath: Can your language model pass chinese elementary school math test? *ArXiv*, abs/2306.16636, 2023.
- Kaiyue Wen, Zhiyuan Li, Jason S. Wang, David Leo Wright Hall, Percy Liang, and Tengyu Ma. Understanding warmup-stable-decay learning rates: A river valley loss landscape perspective. *ArXiv*, abs/2410.05192, 2024.
- Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. *ArXiv*, abs/2203.05482, 2022.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Annual Meeting of the Association for Computational Linguistics*, 2019.
- Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. Agieval: A human-centric benchmark for evaluating foundation models. *ArXiv*, abs/2304.06364, 2023.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *ArXiv*, abs/2311.07911, 2023.
- Wen-Jie Zhou, Bohan Wang, Hongtao Zhang, Chenxi Jia, Wei Chen, and Xueqi Cheng. Extra-merge: Tracing the rank-1 subspace of model merging in language model pre-training. In *International Conference on Machine Learning*, 2026.

A Additional Theory for Hierarchical Merging

This appendix records the modeling detail behind the main-text results. We make the random-effects assumptions of [Sec. 4.2](#) explicit and allow heterogeneous or correlated branches, examine how the selected checkpoint count trades residual reduction against bias, restore the fixed-budget coupling between branch count and branch length, state the variance optimality of the two uniform averages that [Sec. 4.3](#) reports in prose, and give the exact covariance condition it requires. Every loss identity below is exact under the local quadratic model of [Sec. 4.1](#). Throughout Appendices [A](#) and [B](#) we abbreviate the two anchor means of [Sec. 4.2](#) as $\mu_n(K) = \mathbb{E}[\bar{\theta}_n]$ and $m_n(K) = \mathbb{E}[\bar{\theta}_n \mid \mathcal{F}_n]$. To keep the account compact, we develop these extensions as narrative rather than as separate numbered claims, apart from the corollary of Appendix [A.4](#), and give their derivations in place. The three theorems stated in the main text and that corollary are proved in Appendix [B](#).

A.1 Conditional Random Effects and General Covariances

Theorem [4.2](#) uses total covariance without requiring a particular training algorithm. Its shared-shift specialization can be represented by a local random-effects model. For fixed t , assume

$$\mathbb{E}[\theta_{n,(j)} \mid \mathcal{F}_n] = \theta^* + b_{\text{com}} + b_{\text{rec},n} + h_{n,j} + u_n, \quad \mathbb{E}[u_n] = 0, \quad h_{n,1} = 0, \quad (\text{A1})$$

where u_n is measurable with respect to $\mathcal{F}_n$ and common to all selected ranks on that branch, and the deterministic offsets absorb selection-induced changes in mean. Here b_{com} is a finite-horizon offset shared by every branch, $b_{\text{rec},n}$ is the systematic offset of recipe ψ_n , and $h_{n,j}$ is the rank- j contribution to the selection-induced shift. Write $\epsilon_{n,j} = \theta_{n,(j)} - \mathbb{E}[\theta_{n,(j)} \mid \mathcal{F}_n]$ for the conditionally centered fluctuation that [Sec. 4.3](#) describes in words. Averaging over the selected ranks then gives $h_n(K) = K^{-1} \sum_{j=1}^K h_{n,j}$ and $\bar{\epsilon}_n(K) = K^{-1} \sum_{j=1}^K \epsilon_{n,j}$, so the mean displacement described in [Sec. 4.2](#) reads

$$\mu_n(K) - \theta^* = b_{\text{com}} + b_{\text{rec},n} + h_n(K), \quad h_n(1) = 0. \quad (\text{A2})$$

A single u_n shared by all selected ranks is exactly the shared-shift condition that Theorem [4.3](#) assumes: it makes $m_n(K) = \mu_n(K) + u_n$ and therefore $V_{\text{inter},n}(K) = \frac{1}{2} \text{tr}(H \text{Cov}(u_n))$ independent of K , so selection moves the anchor only through $h_n(K)$ and the residual average. This is a modeling assumption; if the conditional shift changes across ranks, the general term $V_{\text{inter},n}(K)$ of Theorem [4.2](#) retains that dependence.

The homogeneous independent-anchor case is what exposes the scaling factors, but a general branch collection needs the full anchor cross-covariance $\Gamma_{nn'}(K) = \text{Cov}(\bar{\theta}_n, \bar{\theta}_{n'})$ together with its scalar form $G_{nn'}(K) = \frac{1}{2} \text{tr}(H \Gamma_{nn'}(K))$. Two properties follow directly. First, $G(K)$ is symmetric because $\Gamma_{n'n}(K) = \Gamma_{nn'}(K)^\top$ and H is symmetric, and it is positive semidefinite because $a^\top G(K) a = \frac{1}{2} \mathbb{E}[\|\sum_n a_n(\bar{\theta}_n - \mu_n(K))\|_H^2] \geq 0$ for every $a \in \mathbb{R}^N$. Second, the covariance of the branch-reweighted model $\theta_\alpha = \sum_n \alpha_n \bar{\theta}_n$, which keeps the uniform temporal average inside every branch, is $\sum_{n,n'} \alpha_n \alpha_{n'} \Gamma_{nn'}(K)$, so Theorem [4.1](#) gives, for fixed nonnegative weights summing to one,

$$\mathbb{E}[\mathcal{L}_Q(\theta_\alpha)] - \mathcal{L}^* = B_\alpha(K) + \alpha^\top G(K) \alpha, \quad B_\alpha(K) = \frac{1}{2} \|\mathbb{E}[\theta_\alpha] - \theta^*\|_H^2. \quad (\text{A3})$$

Setting every weight to $1/N$ turns the quadratic form into a double sum over the branch pairs,

$$\mathbb{E}[\mathcal{L}_Q(\theta_{\text{Traj-Soup}})] - \mathcal{L}^* = B_{\text{soup}}(K) + \frac{1}{2N^2} \sum_{n,n'=1}^N \text{tr}(H \Gamma_{nn'}(K)). \quad (\text{A4})$$

Expanding the anchor as $\bar{\theta}_n = \theta^* + b_{\text{com}} + b_{\text{rec},n} + h_n(K) + u_n + \bar{\epsilon}_n(K)$ resolves each block into three contributions: the covariance of the branch offsets u_n , the covariance of the averaged residuals

$\bar{\epsilon}_n(K)$, and the two cross terms between an offset of one branch and the averaged residual of the other. Because u_n is $\mathcal{F}_n$ -measurable, conditional centering removes the cross terms on the diagonal and leaves $G_{nn}(K) = V_{\text{intra},n}(K) + V_{\text{inter},n}$, while placing no restriction on the cross terms for $n \neq n'$. Independent selected anchors make the off-diagonal $\Gamma_{nn'}(K)$ vanish, which reduces **Eq. (A4)** to the heterogeneous form $B_{\text{soup}}(K) + N^{-2} \sum_{n=1}^N [V_{\text{intra},n}(K) + V_{\text{inter},n}]$. Independence of the full random-effects collections is a stronger condition that makes every off-diagonal block vanish separately.

Equal scalar loss-weighted contributions therefore suffice for Theorem 4.3, and the full covariance matrices may still differ. Independent training followed by selection performed separately within each branch preserves anchor independence when the shared setup is fixed, whereas joint selection or a coupled compatibility decision leaves the off-diagonal blocks in place and **Eq. (A4)** applies. Those blocks also bound what branch averaging can achieve, because a collection with positive average off-diagonal contribution drives the branch term toward that average rather than toward zero as N grows, which is the correlated error floor discussed in Appendix ??.

Weighting the anchors unequally is worthwhile exactly when their contributions differ. For independent anchors with $v_n(K) = V_{\text{intra},n}(K) + V_{\text{inter},n} > 0$, the variance term of **Eq. (A3)** is $\sum_n v_n(K) \alpha_n^2$, and Cauchy–Schwarz applied to $\sum_n (\sqrt{v_n(K)} \alpha_n) v_n(K)^{-1/2} = 1$ gives $\sum_n v_n(K) \alpha_n^2 \geq (\sum_n v_n(K)^{-1})^{-1}$, with equality only when α_n is proportional to $v_n(K)^{-1}$. The unique minimizer is thus the inverse-variance weighting $\alpha_n^{\text{var}}(K) = v_n(K)^{-1} / \sum_{n'} v_{n'}(K)^{-1}$. If instead $G(K)$ is exchangeable with common diagonal v and common off-diagonal c , then $\alpha^\top G(K) \alpha = c + (v - c) \sum_n \alpha_n^2$, where $v - c \geq 0$ follows from positive semidefiniteness on vectors orthogonal to $\mathbf{1}$, so equal weights minimize the variance and do so uniquely when $v > c$. The same exchangeable argument transfers to the residual weights over the K ranks within one branch once the branch covariance matrix is replaced by the rank covariance matrix, while rank-dependent means continue to act through the bias instead. Both statements concern fixed weights, and estimating weights from the same realized checkpoints would require accounting for the joint randomness of weights and anchors.

The effective checkpoint count summarizes the rank covariance in a single scalar, so its interpretation needs both the marginal variation and the cross-checkpoint dependence. For a fixed branch, write $c_{jj'} = \frac{1}{2} \text{tr}(H \text{Cov}(\epsilon_{n,j}, \epsilon_{n,j'}))$ and assume the common rank-1 value $c_{jj} = V_{\text{intra}} > 0$. Expanding the covariance of $\bar{\epsilon}_n(K)$ and taking its loss-weighted trace gives

$$V_{\text{intra},n}(K) = \frac{1}{K^2} \sum_{j,j'=1}^K c_{jj'}, \quad K_{\text{eff}}(K) = \frac{K^2 V_{\text{intra}}}{\sum_{j,j'=1}^K c_{jj'}}. \quad (\text{A5})$$

Since the residuals are mean zero, $c_{jj'} = \frac{1}{2} \mathbb{E}[(H^{1/2} \epsilon_{n,j})^\top (H^{1/2} \epsilon_{n,j'})]$, and Cauchy–Schwarz bounds $|c_{jj'}| \leq \sqrt{c_{jj} c_{j'j'}} = V_{\text{intra}}$. If every entry is nonnegative, the covariance sum lies between KV_{intra} and $K^2 V_{\text{intra}}$, so $1 \leq K_{\text{eff}}(K) \leq K$, with the upper bound attained exactly when the off-diagonal entries vanish and strict as soon as their sum is positive. Independent residuals attain that bound. Negative covariances can instead push $K_{\text{eff}}(K)$ above K , heterogeneous ranks invalidate the equal-marginal premise, and when $V_{\text{intra},n}(K) = 0$ the convention $K_{\text{eff}}(K) = \infty$ returns the correct zero contribution. In such settings the general definition in **Sec. 4.3** remains a variance ratio. In particular, increasing K under validation ranking need not increase $K_{\text{eff}}(K)$, which is what the next subsection turns into a tradeoff.

A.2 The Tradeoff in Selecting More Checkpoints

At fixed N and t , adding a lower-ranked checkpoint changes the mean anchor through $h_n(K)$ and the residual covariance through Eq. (A5). A lower individual validation rank does not by itself determine the loss of the resulting average, so the relevant comparison is between the change in bias and the change in the stochastic contribution. Suppose Theorem 4.3 applies to the same branch collection at K and $K + 1$, with K -independent V_{inter} and reference V_{intra} , and write $\theta_{\text{Traj-Soup}}(K)$ for the soup built from that collection at count K , holding N and t fixed. Applying Eq. (5) at both counts cancels $\mathcal{L}^*$ and the branch term V_{inter}/N , which leaves

$$\begin{aligned} & \mathbb{E}[\mathcal{L}_Q(\theta_{\text{Traj-Soup}}(K+1))] - \mathbb{E}[\mathcal{L}_Q(\theta_{\text{Traj-Soup}}(K))] \\ &= B_{\text{soup}}(K+1) - B_{\text{soup}}(K) + \frac{V_{\text{intra}}}{N} \left[\frac{1}{K_{\text{eff}}(K+1)} - \frac{1}{K_{\text{eff}}(K)} \right]. \end{aligned} \quad (\text{A6})$$

When K_{eff} increases, the stochastic increment is negative, so an additional checkpoint improves the expected loss precisely when that reduction outweighs the bias increment. This is the mechanism behind an interior optimum, and it also leaves room for monotone improvement or monotone decline in other geometries. If $V_{\text{inter},n}(K)$ varies with selection, its averaged increment must be retained, and if the branch cross-covariance varies with selection, the corresponding increment follows from Eq. (A4).

A specific bias model makes a finite optimum explicit. Take the illustrative pair $B_{\text{soup}}(K) = B_{\text{soup}}(1) + \gamma(K-1)^2$ with $\gamma > 0$ and $K_{\text{eff}}(K) = K$, which is a specialization we impose rather than a consequence of validation ranking. For continuous $K \geq 1$ the resulting quadratic surrogate is

$$\Phi_N(K) = B_{\text{soup}}(1) + \gamma(K-1)^2 + \frac{V_{\text{intra}}}{NK} + \frac{V_{\text{inter}}}{N}. \quad (\text{A7})$$

Its derivatives are $\Phi'_N(K) = 2\gamma(K-1) - V_{\text{intra}}/(NK^2)$ and $\Phi''_N(K) = 2\gamma + 2V_{\text{intra}}/(NK^3) > 0$, so Φ_N is strictly convex on $K \geq 1$. Its derivative is negative at $K = 1$ and tends to $+\infty$, so there is a unique continuous minimizer $K_N^* > 1$, characterized by $2\gamma(K_N^*)^2(K_N^* - 1) = V_{\text{intra}}/N$. The map $K \mapsto K^2(K-1)$ is strictly increasing for $K > 1$ because its derivative is $K(3K-2) > 0$, and the right-hand side decreases in N , so the preferred count K_N^* decreases as branches are added. On a bounded interval of feasible counts, strict convexity means an integer optimum lies among the feasible floor and ceiling of the clipped continuous minimizer, and a sparse experimental grid requires comparing the available grid points directly. This specialization explains how additional branches can reduce the marginal value of temporal averaging, and it is consistent with the checkpoint-count shift reported in Sec. 5.3.2. Its predicted count depends on the assumed bias law, the available checkpoints, and the residual correlations, so a K -dependent branch component or a departure from the quadratic model changes the objective.

A.3 The Fixed-Budget Allocation Tradeoff

The comparisons above keep the per-branch horizon fixed. The Limited setting instead fixes the total branch-training token budget T and uses $t = T/N$, and restoring that dependence makes the competition between serial progress and averaging explicit. Let $\mathcal{E}_{\text{Traj-Soup}}(T, N, K)$ denote the expected validation excess loss of $\theta_{\text{Traj-Soup}}(N, T/N, K)$, and suppose Theorem 4.3 holds at every compared horizon with a common reference θ^* and curvature H , which is what makes those excess losses comparable. Writing the branch horizon as an explicit first argument of the bias, of the two

loss-weighted variances, and of the effective count, and substituting $t = T/N$ into **Eq. (5)**, gives

$$\mathcal{E}_{\text{Traj-Soup}}(T, N, K) = B_{\text{soup}}(T/N, K) + \frac{V_{\text{intra}}(T/N)}{NK_{\text{eff}}(T/N, K)} + \frac{V_{\text{inter}}(T/N)}{N} \quad (\text{A8})$$

for any feasible N and K . Increasing N supplies more branches while shortening each one. Shorter horizons may retain greater finite-training bias, averaging may reduce both persistent and residual variation, and checkpoint availability constrains the feasible K . In a serial-saturation regime the cost of shortening a branch can be small enough for the averaging gains to dominate, whereas earlier in training substantial useful serial progress can favor longer branches. This is a loss-accounting identity rather than an allocation rule, since it imposes the budget constraint without supplying the horizon dependence of the individual terms. An optimal N and K therefore requires further information about those functions and about the feasible candidate sets, and the budget counts training tokens, whose relation to matched compute follows the experimental accounting for the compared recipes and architectures.

A.4 Variance-Optimal Weights at Both Levels

Theorem 4.3 takes the two uniform averages of Trajectory Soup as given, which invites the question of whether uniform weights are the right choice or merely a convenient one. For independent errors the answer is classical, since the covariance analysis of averaging assigns equal weight to exchangeable contributions (Bishop, 2006). Transferring that principle to our setting requires care, because the averaged quantities are parameter fluctuations weighted by the validation Hessian, and the checkpoints entering each branch anchor are selected by validation loss rather than drawn independently. SWA also averages iterates uniformly (Izmailov et al., 2018), but for selected checkpoints the optimality of that choice depends on their covariance structure. This subsection makes that dependence explicit and states the result that **Sec. 4.3** reports in prose.

Fix N, K, t , and the checkpoint-selection rule, and replace the two uniform averages by deterministic nonnegative weights satisfying $\sum_{n=1}^N \alpha_n = 1$ and $\sum_{j=1}^K w_{n,j} = 1$ for each branch. The resulting model is $\theta_{\alpha,w} = \sum_{n=1}^N \alpha_n \sum_{j=1}^K w_{n,j} \theta_{n,(j)}$, which recovers $\theta_{\text{Traj-Soup}}$ of **Eq. (2)** at $\alpha_n = 1/N$ and $w_{n,j} = 1/K$. Theorem 4.1 splits its excess loss into

$$\mathbb{E}[\mathcal{L}_Q(\theta_{\alpha,w})] - \mathcal{L}^* = B_{\alpha,w}(K) + \mathcal{V}_{\alpha,w}(K), \quad (\text{A9})$$

where $B_{\alpha,w}(K) = \frac{1}{2} \|\mathbb{E}[\theta_{\alpha,w}] - \theta^*\|_H^2$ and $\mathcal{V}_{\alpha,w}(K) = \frac{1}{2} \text{tr}(H \text{Cov}(\theta_{\alpha,w}))$. These weights act on the covariance of the conditionally centered residuals of Appendix A.1, which we collect into the positive semidefinite matrix $[G_n^e(K)]_{jj'} = \frac{1}{2} \text{tr}(H \text{Cov}(\epsilon_{n,j}, \epsilon_{n,j'}))$. Uniform temporal weights become variance-optimal as soon as every selected checkpoint carries the same aggregate covariance with the selected set. This condition permits correlated residuals and holds for exchangeable covariance, so it is weaker than independence. Throughout, $K_{\text{eff}}(K)$ keeps the definition it received for the uniformly averaged anchor in **Sec. 4.3**.

Corollary A.1 (Variance-optimal averaging at both levels). *Under the homogeneous contributions of Theorem 4.3, assume that each selected rank has the same persistent conditional shift u_n and that the full collections $(u_n, \epsilon_{n,1}, \dots, \epsilon_{n,K})$ are independent across branches. If $G_n^e(K) \mathbf{1}_K = \lambda_n \mathbf{1}_K$ for some scalar λ_n for every branch, then*

$$\mathcal{V}_{\alpha,w}(K) \geq \frac{V_{\text{intra}}}{NK_{\text{eff}}(K)} + \frac{V_{\text{inter}}}{N}. \quad (\text{A10})$$

Uniform temporal and branch weights, $w_{n,j} = 1/K$ and $\alpha_n = 1/N$, attain this bound.

Equal row sums and positive semidefiniteness make uniform temporal weights minimize each residual quadratic form, and the branch-level minimum then follows from $\sum_{n=1}^N \alpha_n^2 \geq 1/N$. Appendix B.4 carries out both steps and prices the two ways of departing from Trajectory Soup separately, charging a quadratic penalty for imbalance across branches and another for unequal temporal weights within them. Because both penalties are nonnegative, the minimum in Eq. (A10) is exactly the stochastic contribution of Eq. (5), so the two uniform averages of our method receive a single joint justification rather than two separate ones. The conclusion nevertheless constrains the variance alone, since changing the temporal weights moves the mean selected displacement and hence $B_{\alpha,w}(K)$, so uniform weights minimize the total loss only when that bias is constant across the admissible weights, as happens when all selected checkpoint means coincide. This remaining gap is what leaves room for deliberately nonuniform temporal weights, while the weighting and allocation ablations of Sec. 5.3.2 settle the practical choice. Heterogeneous or correlated branches, for which inverse-variance branch weights are preferable, are treated in Appendix A.1.

A.5 A General Criterion for Uniform Weighting

The equal-row-sum hypothesis of Corollary A.1 is an instance of a general fact about quadratic forms on the simplex, namely that uniform weights are optimal precisely when every member carries the same aggregate covariance with the collection. Let $A \in \mathbb{R}^{m \times m}$ be symmetric positive semidefinite and let $p = m^{-1}\mathbf{1}_m$. Then p minimizes $a^\top Aa$ over $a_i \geq 0$ with $\mathbf{1}_m^\top a = 1$ if and only if $A\mathbf{1}_m = \lambda\mathbf{1}_m$ for some scalar λ . For the forward direction, write $d = a - p$ so that $\mathbf{1}_m^\top d = 0$, and expand $a^\top Aa = p^\top Ap + 2d^\top Ap + d^\top Ad$. The row-sum condition gives $Ap = (\lambda/m)\mathbf{1}_m$, which annihilates the mixed term and leaves $a^\top Aa = \lambda/m + d^\top Ad$, so positive semidefiniteness establishes optimality and positivity on nonzero zero-sum directions establishes uniqueness. Conversely, if p is optimal, then every entry of p is positive, so $p \pm \tau d$ remains feasible for small $\tau > 0$ along any zero-sum d . The directional derivative at p must therefore vanish, and $d^\top Ap = 0$ for every such d forces Ap to be proportional to $\mathbf{1}_m$.

Applying this criterion to $A = G_n^\epsilon(K)$ gives the within-trajectory condition used in Corollary A.1. For the uniform anchor, $p^\top G_n^\epsilon(K)p = V_{\text{intra},n}(K) = V_{\text{intra}}/K_{\text{eff}}(K)$, so the common row sum must equal $KV_{\text{intra}}/K_{\text{eff}}(K)$. The same criterion applies to the branch covariance matrix $G(K)$ in Eq. (A3), and in particular independent anchors with equal variance contributions have $G(K) = v(K)I_N$ and therefore admit uniform variance-optimal branch weights.

An explicit correlated temporal model shows what the criterion buys. Suppose $K \geq 2$ and $G_n^\epsilon(K) = (V_{\text{intra}} - c_\epsilon)I_K + c_\epsilon\mathbf{1}_K\mathbf{1}_K^\top$ with $0 \leq c_\epsilon \leq V_{\text{intra}}$, which assumes exchangeability of the loss-weighted covariance without requiring identical checkpoint means or independent residuals. Normalization gives $w_n^\top G_n^\epsilon(K)w_n = c_\epsilon + (V_{\text{intra}} - c_\epsilon)\|w_n\|_2^2$, so the bound $\|w_n\|_2^2 \geq 1/K$ shows that uniform weights attain $V_{\text{intra},n}(K) = c_\epsilon + (V_{\text{intra}} - c_\epsilon)/K$, equivalently $K_{\text{eff}}(K) = KV_{\text{intra}}/[V_{\text{intra}} + (K-1)c_\epsilon]$, and do so uniquely when $c_\epsilon < V_{\text{intra}}$. At $c_\epsilon = V_{\text{intra}}$ every normalized weighting carries the same residual variance, so temporal averaging provides no variance reduction at all.

A.6 Scope of the Theoretical Claims

The derivations require the local quadratic model of Sec. 4.1, finite second moments, and a shared parameterization. Theorem 4.3 additionally assumes the shared-shift condition of Appendix A.1, independent selected anchors, and common loss-weighted contributions across branches. The general covariance treatment of Appendix A.1 relaxes the last two of these, retaining the full anchor cross-covariance and allowing heterogeneous branch contributions. These assumptions are

sufficient for the stated identities and are not necessary conditions for empirical merging gains.

The theory addresses population validation loss under Q . It does not directly imply the ordering of discrete benchmark accuracies or post-training outcomes, which are supported by the corresponding experiments. It also does not imply that all recipe deviations are zero-mean noise, that every additional branch improves the model, or that the local quadratic model remains accurate at the displacements the merges actually traverse.

B Proofs of the Main-Text Results

This appendix proves Theorems 4.1–4.3 and Corollary A.1. The extensions of Appendix A are derived where they are stated. All expectations use the fixed setup specified in Sec. 4.1, and every statement is proved under the local quadratic model adopted there, so all identities below are exact. Second moments are assumed finite whenever the corresponding quantities occur. The positive semidefinite matrix H may be singular, so $\|\cdot\|_H$ can be a seminorm. None of the proofs requires H to be invertible.

B.1 Proof of Theorem 4.1

Proof. Let $\mu = \mathbb{E}[\vartheta]$ and write $\vartheta - \theta^* = (\mu - \theta^*) + (\vartheta - \mu)$. Expanding the quadratic form and taking expectations gives

$$\begin{aligned} \mathbb{E}[\|\vartheta - \theta^*\|_H^2] &= \|\mu - \theta^*\|_H^2 + 2(\mu - \theta^*)^\top H \mathbb{E}[\vartheta - \mu] \\ &\quad + \mathbb{E}[(\vartheta - \mu)^\top H (\vartheta - \mu)]. \end{aligned} \quad (\text{A11})$$

The middle term vanishes because $\mathbb{E}[\vartheta - \mu] = 0$. The last term is

$$\mathbb{E}[(\vartheta - \mu)^\top H (\vartheta - \mu)] = \text{tr}\left(H \mathbb{E}[(\vartheta - \mu)(\vartheta - \mu)^\top]\right) = \text{tr}(H \text{Cov}(\vartheta)). \quad (\text{A12})$$

Taking expectations in the local quadratic model of Sec. 4.1 now gives Eq. (3).

The squared bias is nonnegative because $H \succeq 0$. The variance term is nonnegative because it equals $\mathbb{E}[\|H^{1/2}(\vartheta - \mu)\|_2^2]$. The two terms therefore account for the entire expected excess loss under the local quadratic model. $\square$

B.2 Proof of Theorem 4.2

Proof. For fixed n and K , decompose the centered anchor as $\bar{\theta}_n - \mu_n(K) = m_n(K) - \mu_n(K) + \bar{\theta}_n - m_n(K)$. The conditional residual has zero mean given $\mathcal{F}_n$, while $m_n(K) - \mu_n(K)$ is $\mathcal{F}_n$ -measurable. Their mixed second moment is therefore zero. Expanding the covariance yields the law of total covariance,

$$\text{Cov}(\bar{\theta}_n) = \mathbb{E}[\text{Cov}(\bar{\theta}_n \mid \mathcal{F}_n)] + \text{Cov}(m_n(K)). \quad (\text{A13})$$

Substituting this identity into Theorem 4.1 gives Eq. (4). Each covariance contribution is nonnegative after taking its trace against H , by the argument used in Appendix B.1.

If $m_n(K) = \mu_n(K) + u_n$ with the same mean-zero u_n for every feasible K , then $\text{Cov}(m_n(K)) = \text{Cov}(u_n)$. Thus $V_{\text{inter},n}(K) = V_{\text{inter},n} = \frac{1}{2} \text{tr}(H \text{Cov}(u_n))$ is independent of K , which is the shared-shift condition assumed by Theorem 4.3. At $K = 1$, the anchor equals $\theta_{n,(1)}$. Subtracting the $K = 1$ instance of Eq. (4) then cancels both $\mathcal{L}^*$ and $V_{\text{inter},n}$, leaving

$$\mathbb{E}[\mathcal{L}_Q(\bar{\theta}_n)] - \mathbb{E}[\mathcal{L}_Q(\theta_{n,(1)})] = B_n(K) - B_n(1) + V_{\text{intra},n}(K) - V_{\text{intra},n}(1),$$

so merging the top K checkpoints of a branch improves on its best single checkpoint exactly when the intra-trajectory term falls by more than the bias rises. $\square$

B.3 Proof of Theorem 4.3

Proof. Linearity of conditional expectation gives $\bar{\epsilon}_n(K) = \bar{\theta}_n - m_n(K)$. Since $\mathbb{E}[\bar{\epsilon}_n(K) \mid \mathcal{F}_n] = 0$,

$$\text{Cov}(\bar{\epsilon}_n(K)) = \mathbb{E}[\text{Cov}(\bar{\theta}_n \mid \mathcal{F}_n)], \quad \text{Cov}(\bar{\theta}_n) = \text{Cov}(\bar{\epsilon}_n(K)) + \text{Cov}(u_n). \quad (\text{A14})$$

The first identity also establishes $V_{\text{intra},n}(K) = \frac{1}{2} \text{tr}(H \text{Cov}(\bar{\epsilon}_n(K)))$, which is the representation used in Sec. 4.3. Expanding the covariance of the finite residual average gives

$$\text{Cov}(\bar{\epsilon}_n(K)) = \frac{1}{K^2} \sum_{j,j'=1}^K \text{Cov}(\epsilon_{n,j}, \epsilon_{n,j'}). \quad (\text{A15})$$

By linearity of expectation, $\mathbb{E}[\theta_{\text{Traj-Soup}}] = N^{-1} \sum_{n=1}^N \mu_n(K)$, so the term $B_{\text{soup}}(K)$ defined in Theorem 4.3 is the squared curvature-weighted bias of the soup. Eq. (A2) equivalently writes this displacement as

$$\mathbb{E}[\theta_{\text{Traj-Soup}}] - \theta^* = b_{\text{com}} + \frac{1}{N} \sum_{n=1}^N (b_{\text{rec},n} + h_n(K)). \quad (\text{A16})$$

Independence of the selected anchors gives

$$\text{Cov}(\theta_{\text{Traj-Soup}}) = \frac{1}{N^2} \sum_{n=1}^N \text{Cov}(\bar{\theta}_n). \quad (\text{A17})$$

Taking the loss-weighted trace and using the common scalar contributions,

$$\frac{1}{2} \text{tr}(H \text{Cov}(\theta_{\text{Traj-Soup}})) = \frac{1}{N^2} \sum_{n=1}^N \left(\frac{V_{\text{intra}}}{K_{\text{eff}}(K)} + V_{\text{inter}} \right) = \frac{V_{\text{intra}}}{NK_{\text{eff}}(K)} + \frac{V_{\text{inter}}}{N}. \quad (\text{A18})$$

Applying Theorem 4.1 to the soup gives Eq. (5).

For completeness, the rows of Tab. 1 follow from the same identities. A rank-1 checkpoint uses $K_{\text{eff}}(1) = 1$. A branch anchor retains its full persistent branch contribution. A cross-trajectory merge of rank-1 checkpoints sets $K = 1$, and Trajectory Soup uses the full expression. Each operation has its own mean model, so the bias terms differ across rows. $\square$

B.4 Proof of Corollary A.1

Proof. Let $\mu_{n,j} = \mathbb{E}[\theta_{n,(j)}]$. Under the rank-independent conditional shift, $\theta_{n,(j)} = \mu_{n,j} + u_n + \epsilon_{n,j}$ with $\mathbb{E}[\epsilon_{n,j} \mid \mathcal{F}_n] = 0$. Because u_n is $\mathcal{F}_n$ -measurable, $\text{Cov}(u_n, \epsilon_{n,j}) = 0$. The centered weighted anchor is therefore $\sum_{j=1}^K w_{n,j} \theta_{n,(j)} - \sum_{j=1}^K w_{n,j} \mu_{n,j} = u_n + \sum_{j=1}^K w_{n,j} \epsilon_{n,j}$, so its loss-weighted variance equals $w_n^\top G_n^\epsilon(K) w_n + V_{\text{inter}}$. Independence of the branch collections then gives

$$\mathcal{V}_{\alpha,w}(K) = \sum_{n=1}^N \alpha_n^2 \left[w_n^\top G_n^\epsilon(K) w_n + V_{\text{inter}} \right]. \quad (\text{A19})$$

Write $p = K^{-1} \mathbf{1}_K$, $d_n = w_n - p$, and $v(K) = V_{\text{intra}}/K_{\text{eff}}(K) + V_{\text{inter}}$, so that $\mathbf{1}_K^\top d_n = 0$. The equal-row-sum hypothesis gives $G_n^\epsilon(K)p = K^{-1} \lambda_n \mathbf{1}_K$, whence $V_{\text{intra},n}(K) = p^\top G_n^\epsilon(K)p = \lambda_n/K$. By

the definition of $K_{\text{eff}}(K)$ in [Sec. 4.3](#), this quantity equals $V_{\text{intra}}/K_{\text{eff}}(K)$, so the common row sum is $\lambda_n = KV_{\text{intra}}/K_{\text{eff}}(K)$ and $G_n^\epsilon(K)p = (V_{\text{intra}}/K_{\text{eff}}(K))\mathbf{1}_K$. The mixed term $d_n^\top G_n^\epsilon(K)p$ therefore vanishes, and consequently

$$w_n^\top G_n^\epsilon(K)w_n = \frac{V_{\text{intra}}}{K_{\text{eff}}(K)} + d_n^\top G_n^\epsilon(K)d_n. \quad (\text{A20})$$

Normalization of the branch weights yields

$$\sum_{n=1}^N \alpha_n^2 = \frac{1}{N} + \sum_{n=1}^N \left(\alpha_n - \frac{1}{N} \right)^2 \geq \frac{1}{N}, \quad (\text{A21})$$

so substituting [Eq. \(A20\)](#) into [Eq. \(A19\)](#) gives

$$\mathcal{V}_{\alpha,w}(K) = \frac{v(K)}{N} + v(K) \sum_{n=1}^N \left(\alpha_n - \frac{1}{N} \right)^2 + \sum_{n=1}^N \alpha_n^2 d_n^\top G_n^\epsilon(K)d_n. \quad (\text{A22})$$

Positive semidefiniteness of each $G_n^\epsilon(K)$ makes both penalty terms nonnegative. Uniform weights at both levels make them vanish and attain [Eq. \(A10\)](#), and [Theorem 4.1](#) then gives [Eq. \(A9\)](#).

If $v(K) > 0$, equality requires $\alpha_n = 1/N$. If, in addition, every $G_n^\epsilon(K)$ is positive definite on the subspace orthogonal to $\mathbf{1}_K$, equality also requires $w_n = K^{-1}\mathbf{1}_K$ for every branch, so the joint variance minimizer is unique under these conditions. The bias term remains weight-dependent in general. $\square$

C Compute-Scaling and Geometry Protocols

This appendix describes the compute accounting and scaling fits in [Fig. 1b](#), together with the geometric measurements used in [Sec. 2](#) and in the pairwise analysis of [Sec. 5.3.2](#).

C.1 Compute Axis and Scaling-Law Fits

Compute accounting. The horizontal axis shows estimated cumulative mid-training FLOPs on a logarithmic scale, using a sequence length of 262,144 and an estimated cost of 36.74 GFLOPs per token. Token offsets are measured from the shared fork point and summed across branches:

$$C \simeq 3.674 \times 10^{10} D, \quad D = \sum_{n=1}^N t_n, \quad (\text{A23})$$

where t_n is the token horizon of branch n . Equal branch horizons give $D = Nt$. This accounting covers branch training after the shared fork point and excludes the common pretraining prefix. It also leaves out checkpoint storage, the validation passes used for ranking, and the search over how many trajectories and checkpoints to combine, as discussed in [Appendix ??](#). The vertical axis reports the Overall Average Accuracy of [Appendix D.4](#) in percent, without any rescaling.

Scaling fits. Inspired by power-law descriptions of language-model loss ([Kaplan et al., 2020](#); [Hoffmann et al., 2022](#)), we fit an empirical asymptotic inverse power law directly to the accuracy scores:

$$S(C) = A - B(C/C_0)^{-\alpha}, \quad B > 0, \quad \alpha > 0, \quad (\text{A24})$$

where $S(C)$ is the Overall Average Accuracy in percent at cumulative compute C , A is the fitted asymptote, and C_0 is the cumulative FLOPs of the first fitted point on the corresponding curve. The

Raw fit uses $n = 17$ Raw checkpoints from 25B to 425B tokens, and the Trajectory Soup fit uses the $n = 10$ frontier points. The fitted curves are

$$\begin{aligned} S_{\text{raw}}(C) &= 67.2319 - 1.3393 \left(\frac{C}{9.1852 \times 10^{20}} \right)^{-0.45531}, \\ S_{\text{soup}}(C) &= 71.0534 - 2.8580 \left(\frac{C}{1.8370 \times 10^{21}} \right)^{-0.08818}. \end{aligned} \tag{A25}$$

The Raw and Trajectory Soup frontier fits have $R^2 = 0.594$ and $R^2 = 0.965$, respectively. Their reference compute levels C_0 correspond to 25B and 50B processed tokens. These fits summarize the trends in the plotted data.

Displayed range and annotations. The Raw fit is shown as a solid line up to 425B tokens and as a dashed extrapolation beyond that horizon; the extrapolated segment does not enter the fit. At 100B-equivalent compute (3.6741×10^{21} FLOPs), “Quality superiority” compares the fitted accuracies of Trajectory Soup and Raw training, 68.36% and 66.52%, at the same budget. “Scaling robustness” connects the Raw extrapolation at 600B-equivalent compute (2.2044×10^{22} FLOPs, 66.92%) to the observed four-trajectory result at 2400B-equivalent compute (8.8178×10^{22} FLOPs, 69.03%). This annotation highlights the observed improvement from expanding the trajectory budget alongside the regression of later Raw checkpoints.

C.2 Trajectory Directions and Pairwise Alignment

All branches share the architecture and expert layout of Appendix D.1 and are forked from a single checkpoint, so parameters correspond entrywise across branches and no permutation alignment is applied before averaging or measuring angles. For each branch, we first merge checkpoints along its training trajectory and apply PCA to the centered sequence of resulting merged checkpoints over a matched token interval. The approximately rank-one variation of these merged trajectories motivates using the unit leading principal component d_n as the direction of branch n (Zhou et al., 2026), oriented along the net displacement of the merged trajectory over that interval. Because the directions are unit vectors, their pairwise alignment is the cosine $c_{nn'} = d_n^\top d_{n'}$, with associated angle $\phi_{nn'} = \arccos(c_{nn'})$. Fig. 2 reports these cosine similarities for the five compatible branches of Sec. 2. Values near 1 indicate aligned directions, while values near 0 indicate nearly orthogonal directions.

C.3 Interpolation Between Branch Anchors

At each matched token horizon t , we take the merged checkpoints $\bar{\theta}_n(t, K)$ and $\bar{\theta}_{n'}(t, K)$ from two screened branches and interpolate their parameters:

$$\theta_{nn'}(\lambda) = (1 - \lambda)\bar{\theta}_n(t, K) + \lambda\bar{\theta}_{n'}(t, K), \quad \lambda \in [0, 1]. \tag{A26}$$

The endpoints $\lambda \in \{0, 1\}$ are the within-trajectory merged anchors and $\lambda = 1/2$ is their equal-weight cross-trajectory merge. We evaluate $\theta_{nn'}(\lambda)$ on the validation corpus over a grid of λ and retain the best interpolation at each horizon. The left panel of Fig. 3 plots this quantity against consumed tokens, alongside the raw checkpoints of the two branches and their within-trajectory merges; the right panel shows the corresponding interpolation paths in a two-dimensional projection of the weight space, one path per horizon. The comparison therefore measures the additional benefit of combining branch anchors after within-trajectory averaging.

C.4 Pairwise Cosine and Relative Interpolation Gain

The pairwise cosine in Fig. 9 is computed from the same leading PCA directions as in Appendix C.2. Relative gain measures the improvement of the merged model over the midpoint of the two endpoint scores. Let A_n and $A_{n'}$ denote the endpoint scores and $A_{nn'}$ the score after merging the pair. Then $\Delta_{nn'} = A_{nn'} - \frac{1}{2}(A_n + A_{n'})$, reported in points of the overall evaluation score. The subtracted term is the midpoint obtained by linearly interpolating the endpoint scores, so the gain is relative to that reference rather than to an absolute scale, and $\Delta_{nn'} > 0$ means the merge improves on the mean of its endpoints. The scatter relates this gain, averaged over 17 token-budget offsets, to $c_{nn'}$ across branch pairs. Its overlaid line is an ordinary-least-squares fit with a 95% confidence band for the mean response.

Completion fields. For the geometric protocols above, state the intra-trajectory averaging rule and window applied before PCA, the matched token interval and the number of merged points entering each fit, the variance share carried by the leading component, the interpolation grid and the token horizons of Fig. 3, and the 17 token-budget offsets aggregated in Fig. 9. For the compute axis, state the (N, t) configuration behind each plotted Trajectory Soup point and the number of points entering each scaling fit.

D Experimental Configuration

This appendix collects the configuration of the model, the training recipe, the branch identities, and the selection and evaluation protocol. It distinguishes the fields present in the current draft from information still needed for a fully reproducible submission. The measurement conventions for the compute-scaling figure and the geometric observations are given separately in Appendix C. Completion fields are author-facing placeholders and are not additional experimental results.

D.1 Default Model Configuration

The core architecture is a sparse mixture-of-experts (MoE) language model with 7.9B total parameters and 1.3B parameters activated per token. The model has 24 layers, each with 128 routed experts and 8 active routed experts per token, as detailed in Tab. 6.

D.2 Training Settings Shared by All Branches

Every branch is forked from the same pretrained checkpoint and inherits the settings of Tab. 7. The fields along which the branches deliberately differ are held out of this table and listed in Appendix D.3, so that the two tables together specify each recipe exactly.

D.3 Branch Identity and Recipe Mapping

Each branch perturbs the EXP1 baseline along one of the dimensions admitted in Sec. 3.1, except for EXP3, which varies the peak learning rate and the global batch size together. Tab. 8 lists the five recipes, with the perturbed fields of each branch in bold. The default comparison of Sec. 5.2 uses EXP1–EXP3, the four- and five-branch configurations of Fig. 1b and Sec. 5.3.2 add EXP4 and EXP5 in that order, and the geometric analyses of Sec. 2 and Appendix C use all five. All five branches pass the compatibility screen of Sec. 3.1 at the common 600B-token horizon. The smaller-model reproduction of Appendix ?? has its own pair of branches, whose labels are local to that experiment and do not identify the same parameter checkpoints.

Table 6 Default model configuration. The architectural values are taken from the supplied model description and are not independently verified through a public model release.

Field	Supplied value
Model name	Ling-3.0-tiny
Total parameters	7.9B
Active parameters per token	1.3B
Layers	24
Hidden width	1,536
First feedforward layer	Dense
Routed experts in each subsequent layer	128
Active routed experts per token	8
Shared experts per MoE layer	1
Expert feedforward width	512
Attention components	MLA and KDA linear attention
Query LoRA rank	256
KV LoRA rank	512
Vocabulary size	157,184
Tokens consumed before branching	30T
Mid-training sequence length	262,144
Estimated cost per training token	36.74 GFLOPs

D.4 Evaluation Protocol

D.4.0.1 Pre-training evaluation. We evaluate base-model checkpoints before post-training using 41 benchmark configurations across five capability categories: (i) *general knowledge and reasoning*, including ARC-Easy and ARC-Challenge (Clark et al., 2018), AGIEval (Zhong et al., 2023), BBH and its Chinese configuration (Suzgun et al., 2023), WorldSense (Benchekroun et al., 2023), PIQA (Bisk et al., 2020), and HellaSwag (Zellers et al., 2019); (ii) *language understanding*, including RACE-Middle and RACE-High (Lai et al., 2017), SQuAD 2.0 (Rajpurkar et al., 2018), TriviaQA (Joshi et al., 2017), Natural Questions (Kwiatkowski et al., 2019), WinoGrande (Sakaguchi et al., 2019), Belebele (Bandarkar et al., 2024), and CCPM (Li et al., 2021); (iii) *professional knowledge and factuality*,

Table 7 Training settings shared by all branches. Values recorded in the supplied recipe and held fixed across EXP1–EXP5. The perturbed fields are given in Tab. 8.

Field	Value
Complete branch horizon	600B tokens
Optimizer	Muon
Warmup	1% of training tokens
Weight decay	0.1
Gradient clipping	1.0
Optimizer β_1	0.9
Optimizer β_2	0.95
Default checkpoint interval	25B tokens
Dense-sampling ablation interval	12.5B tokens

Table 8 Branch identity and recipe mapping. The five branches forked from the common pretrained checkpoint. Bold entries mark the fields a branch changes relative to the EXP1 baseline; all remaining settings follow Tab. 7.

Branch	Perturbed dimension	Data-shuffling seed	Peak learning rate	Global batch	Schedule after warmup	Muon momentum
EXP1	Baseline	1234	3.39×10^{-4}	256	Constant	0.0
EXP2	Data order	1001	3.39×10^{-4}	256	Constant	0.0
EXP3	Learning rate and batch size	1234	2.40×10^{-4}	128	Constant	0.0
EXP4	Learning-rate schedule	1234	3.39×10^{-4}	256	Decay to 3.39×10^{-5}	0.0
EXP5	Optimizer momentum	1234	3.39×10^{-4}	256	Constant	0.9

including MMLU (Hendrycks et al., 2020), MMLU-Pro (Wang et al., 2024), SuperGPQA (Team et al., 2025), CMMLU (Li et al., 2024a), C-Eval (Huang et al., 2023), SimpleQA (Wei et al., 2024), and Chinese SimpleQA (He et al., 2024); (iv) *mathematics*, including GSM8K (Cobbe et al., 2021), GSM-Plus (Li et al., 2024b), the Chinese subset of MGSM (Shi et al., 2022), MATH and its Minerva evaluation configuration (Hendrycks et al., 2021; Lewkowycz et al., 2022), CMATH (Wei et al., 2023), MathBench (Liu et al., 2024), CollegeMath (Tang et al., 2024), ZKMathUnion, and GKMathUnion; and (v) *coding*, including the English and Chinese configurations of HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), HumanEval+ and MBPP+ (Liu et al., 2023), LiveCodeBench (Jain et al., 2025), CRUXEval (Gu et al., 2024), and BIRD-SQL (Li et al., 2023a).

D.4.0.2 Post-training evaluation. To assess whether mid-training gains persist after post-training, we evaluate models on 16 configurations grouped into six scenarios: (i) *coding*, comprising LiveCodeBench v6 (Jain et al., 2025), SciCode (Tian et al., 2024), and Aider (Aider, 2026); (ii) *mathematics*, comprising HMMT February 2026 (HMMT, 2026), AIME 2025, and AIME 2026 (Mathematical Association of America, 2026); (iii) *reasoning*, comprising KOR-Bench (Ma et al., 2024) and GPQA (Rein et al., 2023); (iv) *knowledge*, comprising MMLU-Pro (Wang et al., 2024), Chinese SimpleQA (He et al., 2024), C-Eval (Huang et al., 2023); (v) *instruction following*, comprising, IFEval (Zhou et al., 2023), IFBench (Pyatkin et al., 2025); and (vi) *function calling*, evaluated using BFCL v4 in function-calling mode (Patil et al., 2025).

D.5 Merging Baselines

The merging scopes compared in Sec. 5.1 all act on the same candidate pools $\mathcal{C}_n(t)$ and all return a uniform average of some subset of those checkpoints, so they differ only in which checkpoints enter the average. We state each one in the notation of Sec. 3.

Single EXP Merge. This baseline merges within one trajectory and performs no cross-trajectory combination, so it is exactly the branch anchor $\bar{\theta}_n(t, K)$ of Eq. (1): the uniform average of the K checkpoints of $\mathcal{C}_n(t)$ that rank highest on the selection objective. Equivalently, it is the $N = 1$ case of Eq. (2), and it therefore exercises only the intra-trajectory term $V_{\text{intra},n}(K)$ of Eq. (4). Every branch produces its own merge; the tables report the strongest of them under the name Single-Trajectory Merge, evaluated at $K = 12$ so that it averages as many checkpoints as the three-branch Trajectory Soup with $K = 4$. Appendix E lists the merges of all branches.

Model Soup. This baseline is the mirror image: it merges across trajectories and performs no within-trajectory selection or averaging. It takes the final checkpoint of each branch at the evaluated

horizon and averages the N of them uniformly,

$$\theta_{\text{Model-Soup}}(N, t) = \frac{1}{N} \sum_{n=1}^N \theta_n(t), \quad (\text{A27})$$

in the spirit of Wortsman et al. (2022), with membership fixed by the admitted branch set rather than chosen by a validation sweep over candidate members. It is not the $K = 1$ case of Eq. (2): that case still ranks each pool and keeps its best element, whereas Eq. (A27) fixes the position at the horizon. Contrasting the two baselines with Trajectory Soup therefore separates the benefit of combining branches from the benefit of selecting what each branch contributes, and the Limited and Extended conventions of Sec. 5.1 determine only the horizon t at which the final checkpoints are taken.

Selection objective. Eq. (1) ranks candidates by validation loss $\hat{\mathcal{L}}_{\text{val}}$. The tables reported here instead rank them by validation accuracy, and the cardinality K of every entry is the best value evaluated on that same objective. Reranking the existing checkpoints by validation loss defines a distinct selection variant whose results would have to come from an actual rerun, since the accuracy-selected tables do not establish that the two selectors agree. Because one validation signal both orders the checkpoints and chooses the reported (N, K) , the selection signal and the reported score are not fully independent.

Completion fields. Specify the validation corpus, token masking, weighting, and loss normalization behind $\hat{\mathcal{L}}_{\text{val}}$, together with the tasks, examples, and aggregation behind the accuracy-based selector and the benchmark weights that form the Overall Average of Appendix D.4. Record the full K grid, the tie-breaking rule, and the split held out for selection, as well as the token position of every reported optimum and the exact Limited horizon of each Model Soup entry. For the post-training comparison, state the SFT data, tokens, optimizer, schedule, and random seeds held fixed across starting checkpoints.

D.6 Ablation Schemes

Every row of Tab. 5 follows the Trajectory Soup (Extended) setting of Tab. 2 and changes exactly one ingredient of Eq. (2): either the coefficients applied inside a branch or the index set those coefficients are applied to. Writing $w_{n,i} \geq 0$ for the weight of checkpoint i in branch n and $\mathcal{I}_n$ for its selected set, with $\sum_{i \in \mathcal{I}_n} w_{n,i} = 1$, every evaluated variant has the form $\frac{1}{N} \sum_{n=1}^N \sum_{i \in \mathcal{I}_n} w_{n,i} \theta_n(i)$. The branch-level weights thus stay uniform throughout, and the ablation varies only the inner stage. The baseline row pairs the EQUAL coefficients with the Top- K each selection, merging $K = 4$ checkpoints from each of the three branches; every other row replaces one of the two components and keeps the other at this setting.

Coefficient schemes. The first group fixes the selected sets to Top- K each and varies the weights inside a branch. Let j denote the within-trajectory rank of a selected checkpoint on the selection objective, with $j = 1$ the best, and normalize each weight vector to sum to one over the K checkpoints of a branch.

EQUAL. Uniform temporal averaging, $w_j = 1/K$, as prescribed by Eq. (2). This is Trajectory Soup itself, and its overall average of 68.96 is the Trajectory Soup (Extended) entry of Tab. 2. Sec. 4.3 identifies it as variance-optimal whenever every selected checkpoint of a branch shares the same aggregate curvature-weighted covariance with its selected set, so the remaining three schemes test how much that condition matters in practice.

1SQRT. Weights $w_j \propto \sqrt{j}$, increasing in the rank index and therefore shifting mass away from the best checkpoints toward the lower-ranked ones. The spread is mild: the K -th member receives $\sqrt{K}$ times the weight of the first.

RANK. Weights $w_j \propto j$, increasing in the same direction but more steeply, with the K -th member receiving K times the weight of the first. Comparing it with 1SQRT separates the direction of the tilt from its magnitude.

RSQRT. Weights $w_j \propto 1/\sqrt{j}$, decreasing in the rank index and thus concentrating mass on the highest-ranked checkpoints. Together with the two increasing schemes it brackets uniform averaging from both sides, so the group probes tilts toward better and toward worse members rather than only one of them.

Selection strategies. The second group fixes the coefficients to EQUAL and varies which elements of the candidate pools $\mathcal{C}_n(t)$ enter the average.

Top- K each. The default rule of Eq. (1), taking $\mathcal{I}_n = \mathcal{I}_n(t, K)$ as the K highest-ranked checkpoints of each branch. It imposes both a quality criterion and a per-branch quota, and the three alternatives below drop one of these two properties each.

Global top- NK . Keeps the quality criterion and removes the quota, taking the NK highest-ranked checkpoints from the pooled candidates of all branches. The merge size matches the baseline, but the branches may be represented unequally, and a branch whose checkpoints score uniformly well can dominate the average. The comparison therefore isolates the value of balanced trajectory representation.

Tail- K per branch. Keeps the quota and the merge size but replaces quality ranking by recency, taking the K latest saved positions of each branch. Since late checkpoints are neither the best nor the most diverse, this isolates the value of ranking candidates at all, as opposed to simply averaging the end of each trajectory.

All checkpoints. Abandons selection entirely, setting $\mathcal{I}_n = \mathcal{C}_n(t)$ for every branch and averaging the pooled candidates uniformly. This is the Full Soup scope of Sec. 5.1, reported here as the no-selection endpoint of the axis. It merges far more checkpoints than the baseline, which makes it the relevant control against the concern that Trajectory Soup benefits merely from averaging more models.

E Per-Trajectory Results of Single-Trajectory Merging

Every branch yields its own Single EXP Merge, the within-trajectory anchor defined in Appendix D.5, and the main-text tables report only the strongest of them under the name Single-Trajectory Merge. This appendix lists all of them. The reported row is chosen separately for each table by its Overall Average, so it need not be the same branch across tables, and it is marked with † below. Because the main text compares Trajectory Soup against the strongest branch, this choice is conservative for the Overall Average, and comparing against any other branch would only widen the reported margins.

E.1 Default Model at Mid-Training

Tab. 9 expands the Single-Trajectory Merge row of **Tab. 2** into the three branches EXP1 (baseline), EXP2 (data shuffle), and EXP3 (learning rate and batch size) of **Tab. 8**. Their Overall Averages span 68.34 to 68.55, with EXP1 the strongest. No branch leads in every category. EXP1 is best on general

Table 9 Single-trajectory merges of every branch on the mid-training evaluation suite. Each row is the Single EXP Merge of one branch of Tab. 2. [†]Reported as Single-Trajectory Merge in Tab. 2. Bold marks the best value in each column.

Base Model	General Knowledge & Reasoning	Language Modeling	Professional Knowledge	Math	Code	Overall Average
EXP1 Merge [†]	62.80	85.86	63.70	72.33	65.36	68.55
EXP2 Merge	62.79	85.44	63.97	72.39	64.99	68.46
EXP3 Merge	62.69	85.65	64.02	71.57	65.34	68.34

Table 10 Single-trajectory merges of every branch after post-training. Each row applies the SFT procedure of Tab. 3 to the Single EXP Merge of one branch. [†]Reported as Single-Trajectory Merge in Tab. 3. Bold marks the best value in each column.

Instruct Model	Math	Code	Knowledge	Reasoning	Instruction Following	Function Call	Overall Average
EXP1 Merge	68.22	47.69	68.98	64.88	57.54	51.75	60.67
EXP2 Merge [†]	68.30	46.58	70.51	62.96	59.46	50.67	61.11
EXP3 Merge	68.38	45.63	69.87	62.40	58.82	51.57	60.67

knowledge and reasoning, language modeling, and code, EXP2 on math, and EXP3 on professional knowledge. Both Trajectory Soup variants of Tab. 2, at 68.72 (Limited) and 68.96 (Extended), exceed all three branches rather than only the reported one.

E.2 Default Model after Post-Training

Tab. 10 applies the SFT procedure of Tab. 3 to each of the three merges above, and the ordering changes. EXP2 now has the highest Overall Average at 61.11, while EXP1 and EXP3 tie at 60.67. The Single-Trajectory Merge rows of Tab. 2 and Tab. 3 therefore correspond to different branches, and the strongest mid-training merge is not necessarily the strongest initialization for SFT. Trajectory Soup reaches 61.39 (Limited) and 61.52 (Extended) in Tab. 3, again above every branch.

E.3 Smaller MoE Model with a WSD Schedule

Tab. 11 lists the two branches of the 2B-parameter reproduction in Tab. 4, where EXP1 is the baseline and EXP2 changes only the data-shuffling seed. These labels are local to that experiment and do not refer to the checkpoints of the default model. The two merges are close, at 49.49 and 49.55. EXP1 is stronger on general knowledge and reasoning, language modeling, and professional knowledge, and EXP2 on math and code. Trajectory Soup reaches 49.64 (Limited) and 50.07 (Extended) and exceeds both.

Table 11 Single-trajectory merges of both branches of the smaller 2B-parameter MoE model. Each row is the Single EXP Merge of one branch of Tab. 4. [†]Reported as Single-Trajectory Merge in Tab. 4. Bold marks the best value in each column.

Model	General Knowledge & Reasoning	Language Modeling	Professional Knowledge	Math	Code	Overall Average
EXP1 Merge	47.97	73.21	43.92	54.09	34.88	49.49
EXP2 Merge [†]	47.71	72.85	43.41	54.21	35.96	49.55